

集成学习算法





目录

01 决策树

02 随机森林

03 Boosting

决策树节点字段的选择

模型介绍

决策树属于经典的十大数据挖掘算法之一，是一种类似于流程图的树结构，其规则就是IF...THEN...的思想，可以用于数值型因变量的预测和离散型因变量的分类。

该算法简单直观、通俗易懂，不需要研究者掌握任何领域知识或复杂的数学推理，而且算法的结果输出具有很强的解释性。

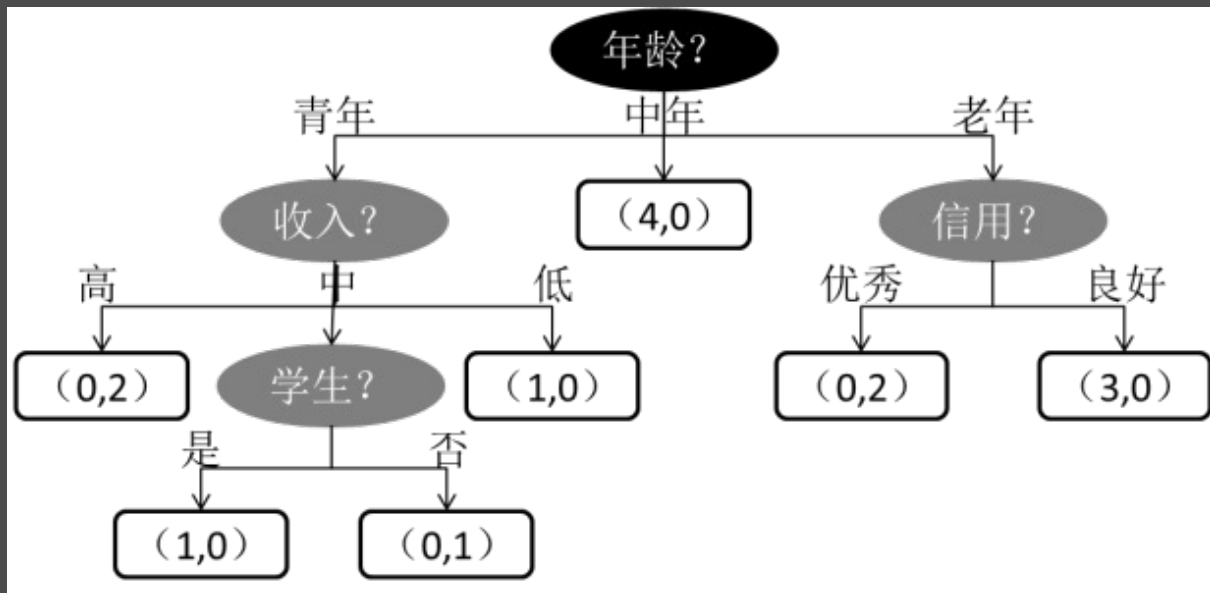
决策树节点字段的选择

模型介绍

Age	Income	Stu	Credit	Buy
青年	高	否	良好	不购买
青年	高	否	优秀	不购买
中年	高	否	良好	购买
老年	中	否	良好	购买
老年	低	是	良好	购买
老年	低	是	优秀	不购买
中年	低	是	优秀	购买
青年	中	否	良好	不购买
青年	低	是	良好	购买
老年	中	是	良好	购买
青年	中	是	优秀	购买
中年	中	否	优秀	购买
中年	高	是	良好	购买
老年	中	否	优秀	不购买

决策树节点字段的选择

模型介绍



图中的决策树呈现自顶向下的生长过程，深色的椭圆表示树的根节点；浅色的椭圆表示树的中间节点；方框则表示树的叶节点。

对于所有的非叶节点来说，都是用来表示条件判断，而叶节点则存储最终的分类结果，例如中年分支下的叶节点 (4,0) 表示4位客户购买，0位客户不购买。

scikit-learn

Machine Learning in Python

[Getting Started](#)
[Release Highlights for 1.5](#)

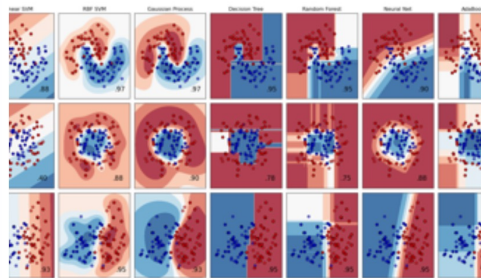
- Simple and efficient tools for predictive data analysis
- Accessible to everybody, and reusable in various contexts
- Built on NumPy, SciPy, and matplotlib
- Open source, commercially usable - BSD license

Classification

Identifying which category an object belongs to.

Applications: Spam detection, image recognition.

Algorithms: [Gradient boosting](#), [nearest neighbors](#), [random forest](#), [logistic regression](#), and [more...](#)



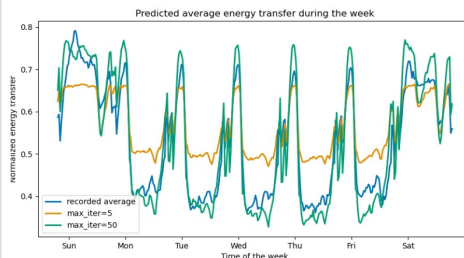
Examples

Regression

Predicting a continuous-valued attribute associated with an object.

Applications: Drug response, stock prices.

Algorithms: [Gradient boosting](#), [nearest neighbors](#), [random forest](#), [ridge](#), and [more...](#)



Examples

Clustering

Automatic grouping of similar objects into sets.

Applications: Customer segmentation, grouping experiment outcomes.

Algorithms: [k-Means](#), [HDBSCAN](#), [hierarchical clustering](#), and [more...](#)



Examples

决策树节点字段的选择

决策树模型的Python函数

```
from sklearn.tree import DecisionTreeClassifier
```

```
DecisionTreeClassifier(criterion='gini', splitter='best',  
                        max_depth=None, min_samples_split=2,  
                        min_samples_leaf=1,  
                        min_weight_fraction_leaf=0.0, max_features=None,  
                        random_state=None, max_leaf_nodes=None,  
                        min_impurity_decrease=0.0, min_impurity_split=None,  
                        class_weight=None, presort=False)
```

criterion: 用于指定选择节点字段的评价指标，对于分类决策树，默认为'gini'，表示采用基尼指数选择节点的最佳分割字段；对于回归决策树，默认为'mse'，表示使用均方误差选择节点的最佳分割字段

splitter: 用于指定节点中的分割点选择方法，默认为'best'，表示从所有的分割点中选择最佳分割点；如果指定为'random'，则表示随机选择分割点

max_depth: 用于指定决策树的最大深度，默认为None，表示树的生长过程中对深度不做任何限制

min_samples_split: 用于指定根节点或中间节点能够继续分割的最小样本量，默认为2

min_samples_leaf: 用于指定叶节点的最小样本量，默认为1

决策树节点字段的选择

决策树模型的Python函数

```
DecisionTreeClassifier(criterion='gini', splitter='best',  
                        max_depth=None, min_samples_split=2,  
                        min_samples_leaf=1,  
                        min_weight_fraction_leaf=0.0, max_features=None,  
                        random_state=None, max_leaf_nodes=None,  
                        min_impurity_decrease=0.0, min_impurity_split=None,  
                        class_weight=None, presort=False)
```

min_weight_fraction_leaf: 用于指定叶节点最小的样本权重，默认为None，表示不考虑叶节点的样本权重

max_features: 用于指定决策树包含的最多分割字段数，默认为None，表示分割时使用所有的字段，与指定'auto'效果一致；如果为具体的整数，则考虑使用对应的分割字段数；如果为0~1的浮点数，则考虑对应百分比的字段个数；如果为'sqrt'，则表示最多考虑 $\sqrt{P}$ 个字段；如果为'log2'，则表示最多使用 $\log_2 P$ 个字段

random_state: 用于指定随机数生成器的种子，默认为None，表示使用默认的随机数生成器

max_leaf_nodes: 用于指定最大的叶节点个数，默认为None，表示对叶节点个数不做任何限制

决策树节点字段的选择

决策树模型的Python函数

```
DecisionTreeClassifier(criterion='gini', splitter='best',  
                        max_depth=None, min_samples_split=2,  
                        min_samples_leaf=1,  
                        min_weight_fraction_leaf=0.0, max_features=None,  
                        random_state=None, max_leaf_nodes=None,  
                        min_impurity_decrease=0.0, min_impurity_split=None,  
                        class_weight=None, presort=False)
```

min_impurity_decrease: 用于指定节点是否继续分割的最小不纯度值，默认为0

min_impurity_split: 同参数min_impurity_decrease含义一致，该参数已在0.21版本剔除

class_weight: 用于指定因变量中类别之间的权重，默认为None，表示每个类别的权重都相等；如果为balanced，则表示类别权重与原始样本中类别的比例成反比；还可以通过字典传递类别之间的权重差异，其形式为{class_label:weight}

presort: bool类型参数，是否对数据进行预排序，默认为False。如果数据集的样本量比较小，设置为True可以提高模型的执行速度；如果数据集的样本量比较大，则不易设置为True

sklearn.tree

Decision tree based models for classification and regression.

User guide. See the [Decision Trees](#) section for further details.

<code>DecisionTreeClassifier</code>	A decision tree classifier.
<code>DecisionTreeRegressor</code>	A decision tree regressor.
<code>ExtraTreeClassifier</code>	An extremely randomized tree classifier.
<code>ExtraTreeRegressor</code>	An extremely randomized tree regressor.

Exporting

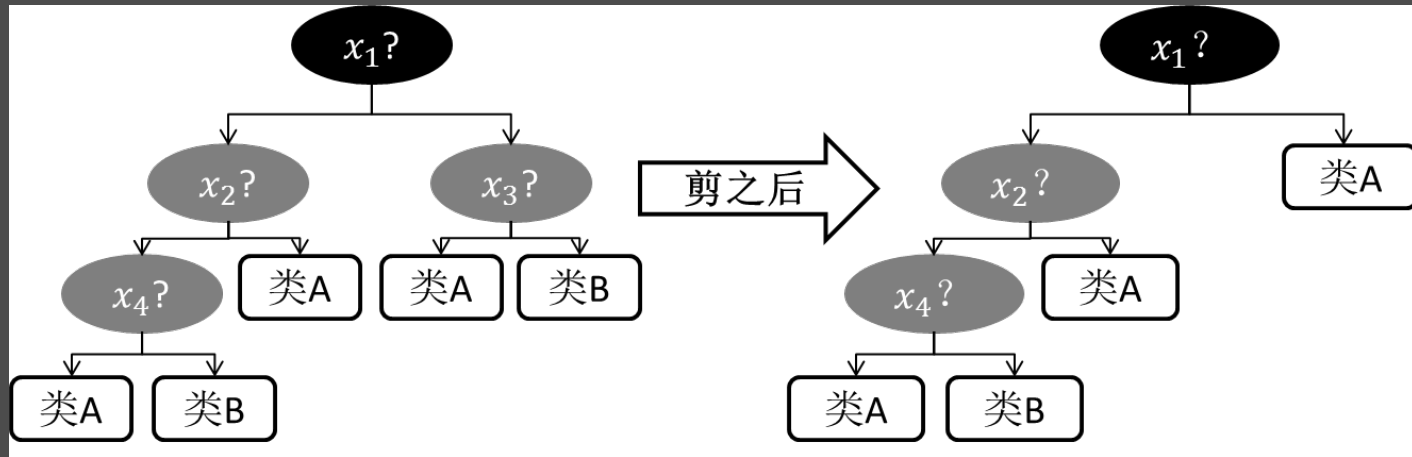
<code>export_graphviz</code>	Export a decision tree in DOT format.
<code>export_text</code>	Build a text report showing the rules of a decision tree.

Plotting

<code>plot_tree</code>	Plot a decision tree.
--	-----------------------

决策树的剪枝技术

误差降低剪枝法



决策树的剪枝技术

误差降低剪枝法

- 将决策树的某个非叶子节点作为剪枝的候选对象（如图中的 x_3 处节点），如果将其子孙节点删除（对应的两个叶节点），则 x_3 处的节点就变成了叶节点。
- 利用投票原则，将此处叶节点中频数最高的类别用作分类标准（如图中剪枝后该叶节点属于类A）。
- 利用剪枝后的新树在测试数据集上进行预测，然后对比新树与老树在测试集上的误判样本量，如果新树的误判样本量低于老树的误判样本量，则将 x_3 处的中间节点替换为叶节点，否则不进行剪枝。
- 重复前面的三步，直到新的决策树能够最大限度地提高测试数据集上的预测准确率。

决策树的剪枝技术

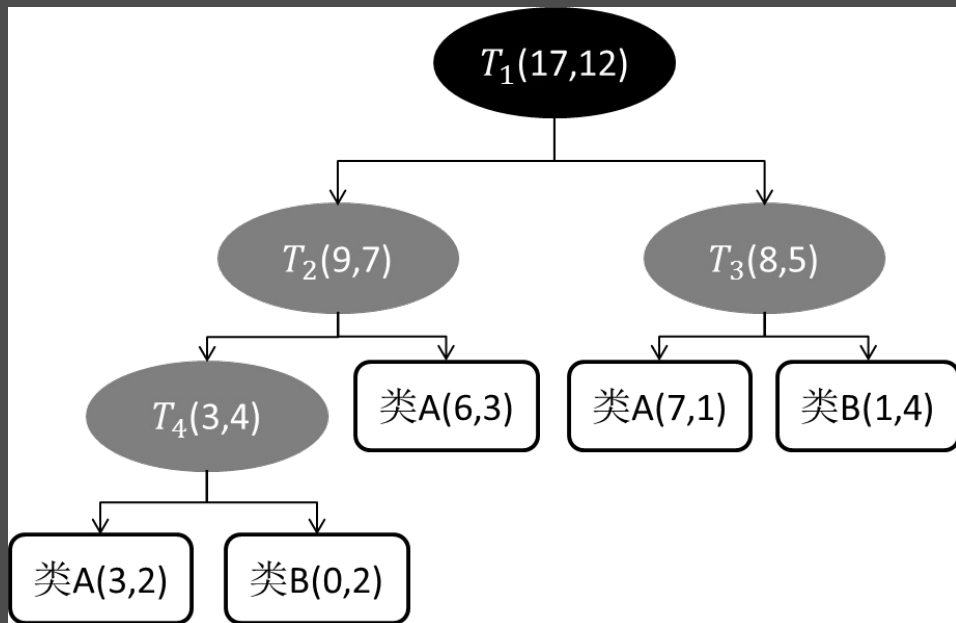
悲观剪枝法

$$\begin{cases} e'(T) = (E(T) + 0.5)/N \\ e'(T_t) = \left(\sum_{i=1}^L (E(t_i) + 0.5) \right) / \left(\sum_{i=1}^L N_i \right) \end{cases}$$

其中, $e'(T)$ 表示剪枝后中间节点 T 被换成叶节点的误判率; $e'(T_t)$ 表示中间节点 T 剪枝前其对应的所有叶节点的误判率; $E(T)$ 为中间节点 T 处的误判个数; $E(t_i)$ 为节点 T 下的所有叶节点误判个数; L 表示中间节点 T 对应的所有叶节点个数; N 表示中间节点 T 的样本个数; N_i 表示各叶节点中的样本个数, 其实 $\sum_{i=1}^L N_i = N$ 。

决策树的剪枝技术

悲观剪枝法



剪枝标准:

对比剪枝前后叶节点误判率的标准就是，如果剪枝后叶节点的误判率期望在剪枝前叶节点误判率期望的一个标准差内，则认为剪枝是合理的，否则不能剪枝。

决策树的剪枝技术

悲观剪枝法

在决策树图中，假设以 T_2 节点为例，剪枝前对应了3个叶节点，误判个数分别为3,2,0；如果将其所有叶节点都剪掉， T_2 便成为了 T_1 的叶节点，误判样本数为7。根据计算公式，可以得到：

$$\begin{cases} e'(T) = \frac{(7 + 0.5)}{16} = 0.469 \\ e'(T_t) = \frac{(3 + 0.5 + 2 + 0.5 + 0 + 0.5)}{(9 + 5 + 2)} = 0.406 \end{cases}$$

$$E(e'(T)) = N \times e'(T) = 16 \times \frac{(7 + 0.5)}{16} = 7.5$$

$$E(e'(T_t)) = N \times e'(T_t) = 6.5$$

$$Var(e'(T_t)) = N \times e'(T_t) \times (1 - e'(T_t)) = 16 \times 0.406 \times (1 - 0.406) = 3.859$$

$$E(e'(T)) < E(e'(T_t)) + \text{Std}(e'(T_t))$$

决策树的剪枝技术

代价复杂度剪枝法

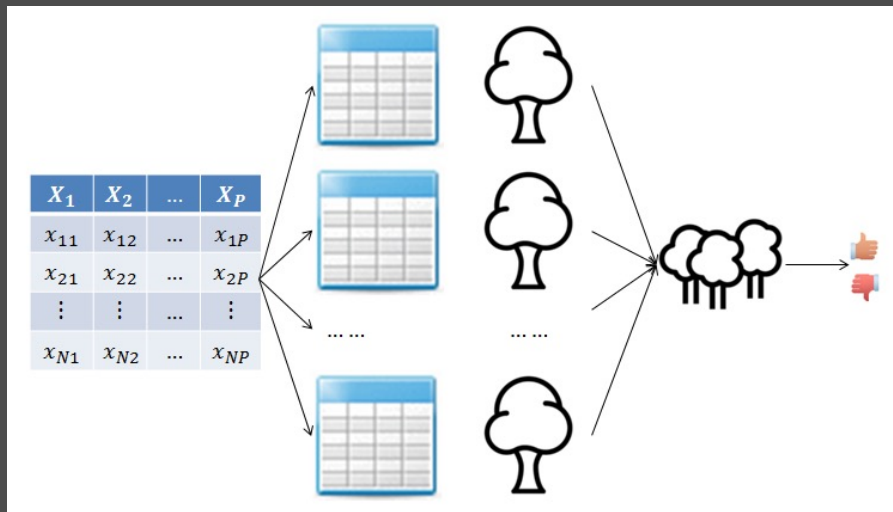
从字面理解，该剪枝方法涉及两则信息，一则是代价，是指将中间节点替换为叶节点后误判率会上升；另一则是复杂度，是指剪枝后叶节点的个数减少，进而使模型的复杂度下降。为了平衡上升的误判率与下降的复杂度，需要加入一个系数 α ，故可以将代价复杂度剪枝法的目标函数写成：

$$C_{\alpha}(T) = C(T) + \alpha \cdot |N_{leaf}|$$

其中， $C(T) = \sum_{i=1}^l N_i \times H(i)$ ； i 表示节点 T 下第 i 个叶节点； N_i 为第 i 个叶节点的样本量； $H(i)$ 为第 i 个叶节点的信息熵； $|N_{leaf}|$ 为节点 T 对应的所有叶节点个数； α 就是调节参数。

随机森林的思想解读

思想解读



利用Bootstrap抽样法，从原始数据集中生成 k 个数据集，并且每个数据集都含有 N 个观测和 P 个自变量。

针对每一个数据集，构造一棵CART决策树，在构建子树的过程中，并没有将所有自变量用作节点字段的选择，而是随机选择 p 个字段。

让每一棵决策树尽可能地充分生长，使得树中的每个节点尽可能“纯净”，即随机森林中的每一棵子树都不需要剪枝。

针对 k 棵CART树的随机森林，对分类问题利用投票法，将最高得票的类别用于最终的判断结果；对回归问题利用均值法，将其用作预测样本的最终结果。

随机森林的思想解读

随机森林的Python函数

```
from sklearn.ensemble import RandomForestClassifier
```

```
RandomForestClassifier(n_estimators=10, criterion='gini', max_depth=None,  
                        min_samples_split=2, min_samples_leaf=1,  
                        min_weight_fraction_leaf=0.0, max_features='auto',  
                        max_leaf_nodes=None, min_impurity_decrease=0.0,  
                        min_impurity_split=None, bootstrap=True,  
                        oob_score=False, n_jobs=1, random_state=None,  
                        verbose=0, warm_start=False, class_weight=None)
```

n_estimators: 用于指定随机森林所包含的决策树个数

criterion: 用于指定每棵决策树节点的分割字段所使用的度量标准，用于分类的随机森林，默认的criterion值为'gini'；用于回归的随机森林，默认的criterion值为'mse'

max_depth: 用于指定每棵决策树的最大深度，默认不限制树的生长深度

min_samples_split: 用于指定每棵决策树根节点或中间节点能够继续分割的最小样本量，默认为2

min_samples_leaf: 用于指定每棵决策树叶节点的最小样本量，默认为1

随机森林的思想解读

随机森林的Python函数

```
RandomForestClassifier(n_estimators=10, criterion='gini', max_depth=None,  
                        min_samples_split=2, min_samples_leaf=1,  
                        min_weight_fraction_leaf=0.0, max_features='auto',  
                        max_leaf_nodes=None, min_impurity_decrease=0.0,  
                        min_impurity_split=None, bootstrap=True,  
                        oob_score=False, n_jobs=1, random_state=None,  
                        verbose=0, warm_start=False, class_weight=None)
```

min_weight_fraction_leaf: 用于指定每棵决策树叶节点最小的样本权重，默认为None，表示不考虑叶节点的样本权重

max_features: 用于指定每棵决策树包含的最多分割字段数，默认为None，表示分割时使用所有的字段

max_leaf_nodes: 用于指定每棵决策树最大的叶节点个数，默认为None，表示对叶节点个数不做任何限制

min_impurity_decrease: 用于指定每棵决策树的节点是否继续分割的最小不纯度值，默认为0

bootstrap: bool类型参数，是否对原始数据集进行bootstrap抽样，用于子树的构建，默认为True

随机森林的思想解读

随机森林的Python函数

```
RandomForestClassifier(n_estimators=10, criterion='gini', max_depth=None,  
                        min_samples_split=2, min_samples_leaf=1,  
                        min_weight_fraction_leaf=0.0, max_features='auto',  
                        max_leaf_nodes=None, min_impurity_decrease=0.0,  
                        min_impurity_split=None, bootstrap=True,  
                        oob_score=False, n_jobs=1, random_state=None,  
                        verbose=0, warm_start=False, class_weight=None)
```

oob_score: bool类型参数, 是否使用包外样本计算泛化误差, 默认为False, 包外样本是指每次bootstrap抽样时没有被抽中的样本

n_jobs: 用于指定计算随机森林算法的CPU个数, 默认为1

random_state: 用于指定随机数生成器的种子, 默认为None, 表示使用默认的随机数生成器

verbose: 用于指定随机森林计算过程中是否输出日志信息, 默认为0, 表示不输出

warm_start: bool类型参数, 是否基于上一次的训练结果进行本次的运算, 默认为False

class_weight: 用于指定因变量中类别之间的权重, 默认为None, 表示每个类别的权重都相等

sklearn.ensemble

Ensemble-based methods for classification, regression and anomaly detection.

User guide. See the [Ensembles: Gradient boosting, random forests, bagging, voting, stacking](#) section for further details.

<code>AdaBoostClassifier</code>	An AdaBoost classifier.
<code>AdaBoostRegressor</code>	An AdaBoost regressor.
<code>BaggingClassifier</code>	A Bagging classifier.
<code>BaggingRegressor</code>	A Bagging regressor.
<code>ExtraTreesClassifier</code>	An extra-trees classifier.
<code>ExtraTreesRegressor</code>	An extra-trees regressor.
<code>GradientBoostingClassifier</code>	Gradient Boosting for classification.
<code>GradientBoostingRegressor</code>	Gradient Boosting for regression.
<code>HistGradientBoostingClassifier</code>	Histogram-based Gradient Boosting Classification Tree.
<code>HistGradientBoostingRegressor</code>	Histogram-based Gradient Boosting Regression Tree.
<code>IsolationForest</code>	Isolation Forest Algorithm.
<code>RandomForestClassifier</code>	A random forest classifier.
<code>RandomForestRegressor</code>	A random forest regressor.

决策树与随机森林的应用实战

Titanic幸存分类--决策树

Titanic										
PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin
1	0	3	Braund, Mr. Owen Harris	male	22	1	0	A/5 21171	7.25	
2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Thayer)	female	38	1	0	PC 17599	71.2833	C85
3	1	3	Heikkinen, Miss. Laina	female	26	0	0	STON/O2. 3101282	7.925	
4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35	1	0	113803	53.1	C123
5	0	3	Allen, Mr. William Henry	male	35	0	0	373450	8.05	
6	0	3	Moran, Mr. James	male		0	0	330877	8.4583	
7	0	1	McCarthy, Mr. Timothy J	male	54	0	0	17463	51.8625	E46
8	0	3	Palsson, Master. Gosta Leonard	male	2	3	1	349909	21.075	
9	1	3	Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)	female	27	0	2	347742	11.1333	
10	1	2	Nasser, Mrs. Nicholas (Adele Achem)	female	14	1	0	237736	30.0708	
11	1	3	Sandstrom, Miss. Marguerite Rut	female	4	1	1	PP 9549	16.7	G6
12	1	1	Bonnell, Miss. Elizabeth	female	58	0	0	113783	26.55	C103
13	0	3	Saunderscock, Mr. William Henry	male	20	0	0	A/5. 2151	8.05	
14	0	3	Andersson, Mr. Anders Johan	male	39	1	5	347082	31.275	
15	0	3	Vestrom, Miss. Hulda Amanda Adolfina	female	14	0	0	350406	7.8542	
16	1	2	Hewlett, Mrs. (Mary D Kingcome)	female	55	0	0	248706	16	
17	0	3	Rice, Master. Eugene	male	2	4	1	382652	29.125	
18	1	2	Williams, Mr. Charles Eugene	male		0	0	244373	13	
19	0	3	Vander Planke, Mrs. Julius (Emelia Maria Vandemoortele)	female	31	1	0	345763	18	

决策树与随机森林的应用实战

Titanic幸存分类--决策树

```
import pandas as pd
# 读入数据
Titanic = pd.read_csv(r'Titanic.csv')
Titanic.head()

# 删除无意义的变量, 并检查剩余自字是否含有缺失值
Titanic.drop(['PassengerId','Name','Ticket','Cabin'], axis
= 1, inplace = True)

Titanic.isnull().sum(axis = 0)

# 对Sex分组, 用各组乘客的平均年龄填充组中的缺失年龄
fillna_Titanic = []
for i in Titanic.Sex.unique():
    update = Titanic.loc[Titanic.Sex == i].fillna(value =
{'Age': Titanic.Age[Titanic.Sex == i].mean()})
    fillna_Titanic.append(update)

Titanic = pd.concat(fillna_Titanic)
```

PassengerId	Survived	Pclass	\
0	1	0	3
1	2	1	1
2	3	1	3
3	4	1	1
4	5	0	3

	Name	Sex	Age	SibSp	\
0	Braund, Mr. Owen Harris	male	22.0	1	
1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	
2	Heikkinen, Miss. Laina	female	26.0	0	
3	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	
4	Allen, Mr. William Henry	male	35.0	0	

Parch	Ticket	Fare	Cabin	Embarked
0	A/5 21171	7.2500	NaN	S
1	PC 17599	71.2833	C85	C
2	STON/O2. 3101282	7.9250	NaN	S
3	113803	53.1000	C123	S
4	373450	8.0500	NaN	S

Survived	0
Pclass	0
Sex	0
Age	177
SibSp	0
Parch	0
Fare	0
Embarked	2
dtype:	int64

决策树与随机森林的应用实战

Titanic幸存分类--决策树

```
# 使用Embarked变量的众数填充缺失值
Titanic.fillna(value =
{'Embarked':Titanic.Embarked.mode()[0]}, inplace=True)

# 将数值型的Pclass转换为类别型, 否则无法对其哑变量处理
Titanic.Pclass = Titanic.Pclass.astype('category')

# 哑变量处理
dummy =
pd.get_dummies(Titanic[['Sex','Embarked','Pclass']])

# 水平合并Titanic数据集和哑变量的数据集
Titanic = pd.concat([Titanic,dummy], axis = 1)

# 删除原始的Sex、Embarked和Pclass变量
Titanic.drop(['Sex','Embarked','Pclass'], inplace=True,
axis = 1)
Titanic.head()
```

```
dummy=
Sex_female Sex_male Embarked_C Embarked_Q Embarked_S Pclass_1 \
0 0 1 0 0 1 0
4 0 1 0 0 1 0
5 0 1 0 1 0 0
6 0 1 0 0 1 1
7 0 1 0 0 1 0

Pclass_2 Pclass_3
0 0 1
4 0 1
5 0 1
6 0 0
7 0 1

Titanic=
Survived Age SibSp Parch Fare Sex_female Sex_male \
0 0 22.000000 1 0 7.2500 0 1
4 0 35.000000 0 0 8.0500 0 1
5 0 30.726645 0 0 8.4583 0 1
6 0 54.000000 0 0 51.8625 0 1
7 0 2.000000 3 1 21.0750 0 1

Embarked_C Embarked_Q Embarked_S Pclass_1 Pclass_2 Pclass_3
0 0 0 1 0 0 1
4 0 0 1 0 0 1
5 0 1 0 0 0 1
6 0 0 1 1 0 0
7 0 0 1 0 0 1
```


决策树与随机森林的应用实战

Titanic幸存分类--决策树

```
# 导入第三方包
from sklearn import model_selection

# 取出所有自变量名称
predictors = Titanic.columns[1:]

# 将数据集拆分为训练集和测试集，且测试集的比例为25%
X_train, X_test, y_train, y_test =
model_selection.train_test_split(Titanic[predictors],
Titanic.Survived, test_size = 0.25, random_state = 1234)

# 通过网格搜索确定参数
# 导入第三方模块
from sklearn.model_selection import GridSearchCV
from sklearn import tree

# 预设各参数的不同选项值
max_depth = [2,3,4,5,6]
min_samples_split = [2,4,6,8]
min_samples_leaf = [2,4,8,10,12]
```

```
# 将各参数值以字典形式组织起来
parameters = {'max_depth':max_depth,
'min_samples_split':min_samples_split,
'min_samples_leaf':min_samples_leaf}

# 网格搜索法，测试不同的参数值
grid_dtcateg = GridSearchCV(estimator =
tree.DecisionTreeClassifier(), param_grid = parameters,
cv=10)

# 模型拟合
grid_dtcateg.fit(X_train, y_train)

# 返回最佳组合的参数值
grid_dtcateg.best_params_
```

out:

```
{'max_depth': 3, 'min_samples_leaf': 4,
'min_samples_split': 2}
```

决策树与随机森林的应用实战

Titanic幸存分类--决策树

```
# 构建分类决策树
CART_Class = tree.DecisionTreeClassifier(max_depth=3, min_samples_leaf=4, min_samples_split=2)
# 模型拟合
decision_tree = CART_Class.fit(X_train, y_train)
# 模型在测试集上的预测
pred = CART_Class.predict(X_test)
# 模型的准确率
print('模型在测试集的预测准确率: \n', metrics.accuracy_score(y_test, pred))
```

out:

模型在测试集的预测准确率:
0.829596412556

决策树与随机森林的应用实战

Titanic幸存分类--决策树

```
# 导入第三方包
import matplotlib.pyplot as plt

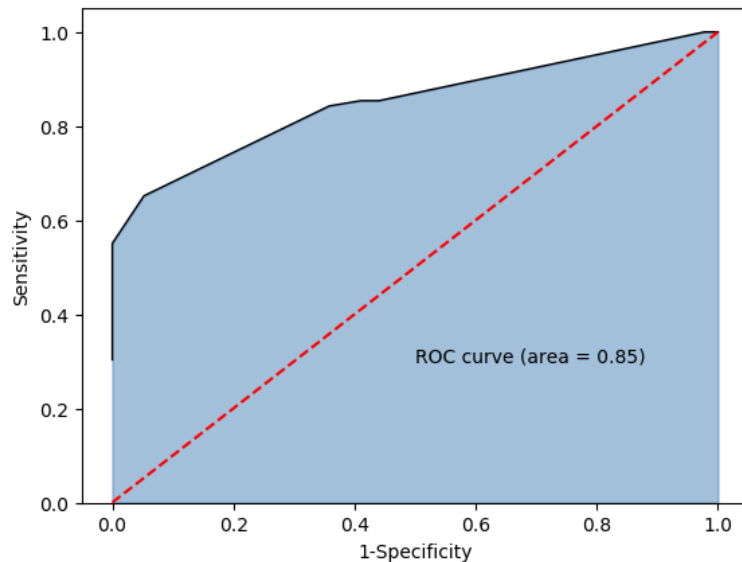
y_score = CART_Class.predict_proba(X_test)[:,-1]
fpr,tpr,threshold = metrics.roc_curve(y_test, y_score)

# 计算AUC的值
roc_auc = metrics.auc(fpr,tpr)

# 绘制面积图
plt.stackplot(fpr, tpr, color='steelblue', alpha = 0.5,
edgecolor = 'black')

# 添加边际线
plt.plot(fpr, tpr, color='black', lw = 1)
# 添加对角线
plt.plot([0,1],[0,1], color = 'red', linestyle = '--')
# 添加文本信息
plt.text(0.5,0.3,'ROC curve (area = %0.2f)' % roc_auc)
# 添加x轴与y轴标签
```

```
plt.xlabel('1-Specificity')
plt.ylabel('Sensitivity')
# 显示图形
plt.show()
```



决策树与随机森林的应用实战

Titanic幸存分类—随机森林

```
# 导入第三方包
from sklearn import ensemble

# 构建随机森林
RF_class = ensemble.RandomForestClassifier(n_estimators=200, random_state=1234)

# 随机森林的拟合
RF_class.fit(X_train, y_train)

# 模型在测试集上的预测
RFclass_pred = RF_class.predict(X_test)

# 模型的准确率
print('模型在测试集的预测准确率: \n', metrics.accuracy_score(y_test, RFclass_pred))
```

out:

模型在测试集的预测准确率:
0.838565022422

决策树与随机森林的应用实战

Titanic幸存分类—随机森林

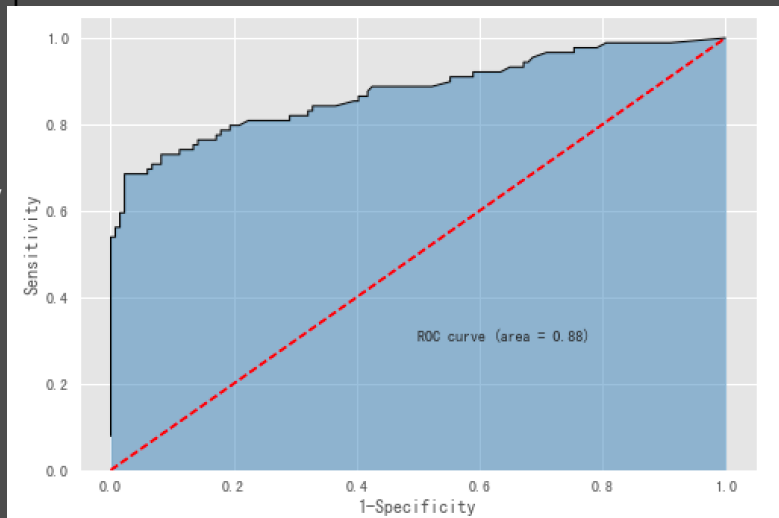
```
# 计算绘图数据
y_score = RF_class.predict_proba(X_test)[:,-1]

fpr,tpr,threshold = metrics.roc_curve(y_test, y_score)

roc_auc = metrics.auc(fpr,tpr)

# 绘图

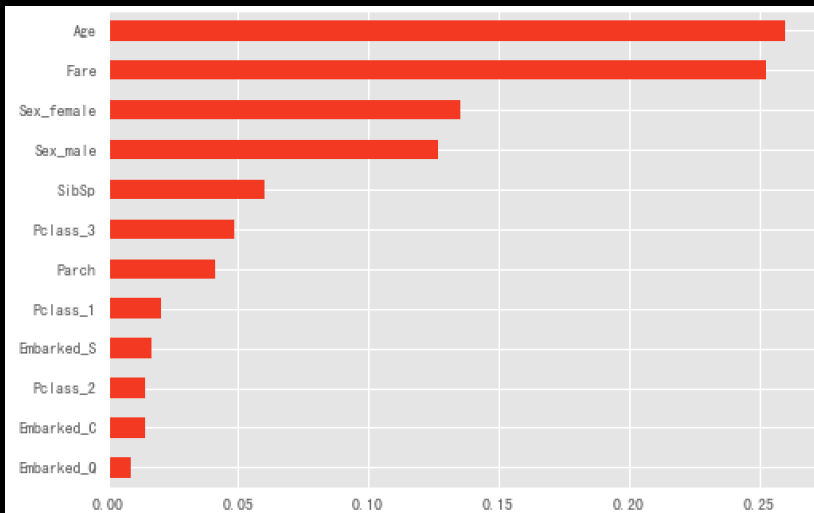
plt.stackplot(fpr, tpr, color='steelblue', alpha = 0.5,
edgecolor = 'black')
plt.plot(fpr, tpr, color='black', lw = 1)
plt.plot([0,1],[0,1], color = 'red', linestyle = '--')
plt.text(0.5,0.3,'ROC curve (area = %0.2f)' % roc_auc)
plt.xlabel('1-Specificity')
plt.ylabel('Sensitivity')
plt.show()
```



决策树与随机森林的应用实战

Titanic幸存分类--随机森林

```
# 变量的重要性程度值
importance = RF_class.feature_importances_
# 构建含序列用于绘图
Impt_Series = pd.Series(importance, index = X_train.columns)
# 对序列排序绘图
Impt_Series.sort_values(ascending = True).plot('barh')
# 显示图形
plt.show()
```



决策树与随机森林的应用实战

肾小球滤过率--决策树

	age_month	sex	black	BMI	HDL	CKD_stage	S_Creat	cal_creat	ls_not_hc	D_epi_eGFR
1	472	1	0	30.22	35	0	1	1	2	94.38848
2	283	1	1	29.98	43	0	1.1	1.1	1	109.0864
3	1011	2	0	24.62	51	0	0.8	0.8	1	67.70044
4	176	2	0	27.28	48	1	0.6	0.6	3	136.8617
5	534	1	0	33.84	37	0	0.9	0.9	2	103.5109
6	762	2	0	26.6	53	0	0.7	0.7	0	92.2082
7	156	1	1	25.18	57	0	0.9	0.9	1	149.1567
8	436	1	0	22.51	93	0	1	1	1	96.39872
9	193	1	0	21.12	41	0	0.7	0.7	2	139.7222
10	160	1	0	15.19	66	0	0.5	0.5	4	163.8613
11	559	2	0	36.07	29	0	0.7	0.7	4	103.904
12	648	2	0	31.53	52	0	0.7	0.7	1	98.22597
13	200	1	0	28.44	49	0	0.7	0.7	1	139.7222
14	337	1	1	25.2	64	0	1.3	1.3	3	86.06037
15	346	1	0	24.54	42	0	0.8	0.8	2	121.5687
16	211	1	0	19.69	61	0	0.8	0.8	2	131.3349
17	559	1	0	29.64	35	1	0.9	0.9	12	102.0668
18	468	2	1	32.12	39	1	0.9	0.9	5	93.35022
19	767	2	0	26.57	61	1	0.7	0.7	3	92.2082
20	173	2	1	20.05	45	0	0.8	0.8	1	128.3003
21	703	1	0	29.52	39	0	0.8	0.8	5	98.46893

决策树与随机森林的应用实战

肾小球滤过率--决策树

```
# 读入数据
NHANES = pd.read_excel(r'NHANES.xlsx')

# 取出自变量名称
predictors = NHANES.columns[:-1]

# 将数据集拆分为训练集和测试集
X_train, X_test, y_train, y_test =
model_selection.train_test_split(NHANES[predictors],
NHANES.CKD_epi_eGFR, test_size = 0.25, random_state
= 1234)

# 预设各参数的不同选项值
max_depth = [18,19,20,21,22]
min_samples_split = [2,4,6,8]
min_samples_leaf = [2,4,8]
parameters = {'max_depth':max_depth,
'min_samples_split':min_samples_split,
'min_samples_leaf':min_samples_leaf}
```

```
# 网格搜索法, 测试不同的参数值
grid_dtreg = GridSearchCV(estimator =
tree.DecisionTreeRegressor(), param_grid = parameters,
cv=10)
```

```
# 模型拟合
grid_dtreg.fit(X_train, y_train)
```

```
# 返回最佳组合的参数值
grid_dtreg.best_params_
```

out:
{ 'max_depth' : 21, 'min_samples_leaf' : 2,
'min_samples_split' : 4 }

决策树与随机森林的应用实战

肾小球滤过率--决策树

```
# 构建用于回归的决策树
CART_Reg = tree.DecisionTreeRegressor(max_depth = 21, min_samples_leaf = 2, min_samples_split = 4)

# 回归树拟合
CART_Reg.fit(X_train, y_train)

# 模型在测试集上的预测
pred = CART_Reg.predict(X_test)

# 计算衡量模型好坏的MSE值
metrics.mean_squared_error(y_test, pred)

out:
1.8719091371521341
```

决策树与随机森林的应用实战

肾小球滤过率--随机森林

```
# 构建用于回归的随机森林
```

```
RF = ensemble.RandomForestRegressor(n_estimators=200, random_state=1234)
```

```
# 随机森林拟合
```

```
RF.fit(X_train, y_train)
```

out:

0.89592317816584133

```
# 模型在测试集上的预测
```

```
RF_pred = RF.predict(X_test)
```

```
# 计算模型的MSE值
```

```
metrics.mean_squared_error(y_test, RF_pred)
```

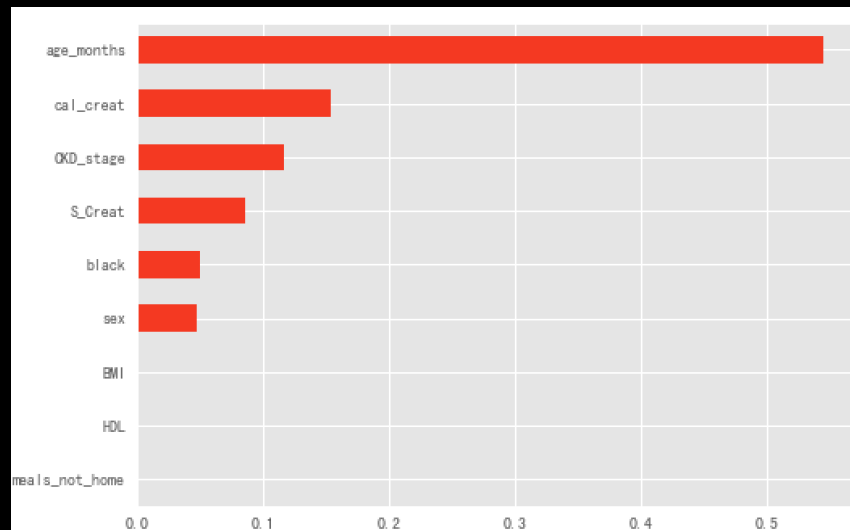
```
# 构建变量重要性的序列
```

```
importance = pd.Series(RF.feature_importances_,  
                        index = X_train.columns)
```

```
# 排序并绘图
```

```
importance.sort_values().plot('barh')
```

```
plt.show()
```



提升树之Adaboost算法的介绍

模型介绍

提升树算法与线性回归模型的思想类似，所不同的是该算法实现了多棵基础决策树 $f(x)$ 的加权运算，最具代表的提升树为AdaBoost算法，即：

$$F(x) = \sum_{m=1}^M \alpha_m f_m(x) = F_{m-1}(x) + \alpha_m f_m(x)$$

其中， $F(x)$ 是由 M 棵基础决策树构成的最终提升树， $F_{m-1}(x)$ 表示经过 $m-1$ 轮迭代后的提升树， α_m 为第 m 棵基础决策树所对应的权重， $f_m(x)$ 为第 m 棵基础决策树。

提升树之Adaboost算法的介绍

损失函数的介绍

对于Adaboost算法而言，每一棵基础决策树都是基于前一棵基础决策树的分类结果对样本点设置不同的权重，如果在前一棵基础决策树中将某样本点预测错误，就会增大该样本点的权重，否则会相应降低样本点的权重，进而再构建下一棵基础决策树，更加关注权重大的样本点。

所以，AdaBoost算法需要解决三大难题，即样本点的权重 w_{mi} 如何确定、基础决策树 $f(x)$ 如何选择以及每一棵基础决策树所对应的权重 α_m 如何计算。

提升树之Adaboost算法的介绍

损失函数的介绍

$$\begin{aligned} L(y, F(x)) &= \exp(-yF(x)) \\ &= \exp\left(-y \sum_{m=1}^M \alpha_m f_m(x)\right) \\ &= \exp\left(-y(F_{m-1}(x) + \alpha_m f_m(x))\right) \end{aligned}$$

如果提升树 $F_{m-1}(x)$ 还能够继续提升, 就说明损失函数还能够继续降低, 换句话说, 如果将所有训练样本点带入损失函数中, 一定存在一个最佳的 α_m 和 $f_m(x)$, 使得损失函数尽量最大化地降低, 即:

$$(\alpha_m, f_m(x)) = \operatorname{argmin}_{\alpha, f(x)} \sum_{i=1}^N \exp\left(-y_i(F_{m-1}(x_i) + \alpha_m f_m(x_i))\right)$$

提升树之Adaboost算法的介绍

损失函数的介绍

进一步，还可以将最小化的目标函数改写成下式：

$$(\alpha_m, f_m(x)) = \operatorname{argmin}_{\alpha, f(x)} \sum_{i=1}^N p_{mi} \exp(-y_i \alpha_m f_m(x_i))$$

其中, $p_{mi} = \exp[-y_i F_{m-1}(x_i)]$, 由于 p_{mi} 与损失函数中的 α_m 和 $f_m(x)$ 无关, 因此在求解最小化的问题时只需重点关注 $\sum_{i=1}^N \exp(-y_i \alpha_m f_m(x_i))$ 部分。

提升树之Adaboost算法的介绍

损失函数的介绍

对于 $\sum_{i=1}^N \exp(-y_i \alpha_m f_m(x_i))$ 而言, 当第 m 棵基础决策树能够准确预测时, y_i 与 $f_m(x_i)$ 的乘积为1, 否则为-1, 于是 $\exp(-y_i \alpha_m f_m(x_i))$ 的结果为 $\exp(-\alpha_m)$ 或 $\exp(\alpha_m)$, 对于某个固定的 α_m 而言, 损失函数中的和式仅仅是关于 α_m 的式子。所以, 要想求得损失函数的最小值, 首先得找到最佳的 $f_m(x)$, 使得所有训练样本点 x_i 带入 $f_m(x)$ 后, 误判结果越少越好, 即最佳的 $f_m(x)$ 可以表示为:

$$f_m(x)^* = \operatorname{argmin}_f \sum_{i=1}^N p_{mi} I(y_i \neq f_m(x))$$

其中, f 表示所有可用的基础决策树空间, $f_m(x)^*$ 就是从 f 空间中寻找到的第 m 轮基础决策树, 它能够使加权训练样本点的分类错误率最小, $I(y_i \neq f_m(x))$ 表示当第 m 棵基础决策树预测结果与实际值不相等时返回1。

提升树之Adaboost算法的介绍

损失函数的介绍

$$\begin{aligned} L(y, F(x)) &= \exp(-y(F_{m-1}(x) + \alpha_m f_m(x))) \\ &= \sum_{i=1}^N p_{mi} \exp(-y_i \alpha_m f_m(x_i)) \\ &= \sum_{y_i=f_m(x_i)} p_{mi} \exp(-\alpha_m) + \sum_{y_i \neq f_m(x_i)} p_{mi} \exp(\alpha_m) \\ &= (\exp(\alpha_m) - \exp(-\alpha_m)) \sum_{i=1}^N p_{mi} I(y_i \neq f_m(x)) \\ &\quad + \exp(-\alpha_m) \sum_{i=1}^N p_{mi} \end{aligned}$$

提升树之Adaboost算法的介绍

损失函数的介绍

$$\begin{aligned}\sum_{i=1}^N p_{mi} &= \sum_{i=1}^N p_{mi} I(y_i \neq f_m(x)) + \sum_{i=1}^N p_{mi} I(y_i = f_m(x)) \\ &= \sum_{y_i \neq f_m(x_i)} p_{mi} + \sum_{y_i = f_m(x_i)} p_{mi}\end{aligned}$$

✚ 求偏导，令导函数为0

$$\frac{\partial L(y, F(x))}{\partial \alpha_m} = (\alpha_m e^{\alpha_m} + \alpha_m e^{-\alpha_m}) \sum_{i=1}^N p_{mi} I(y_i \neq f_m(x)) - \alpha_m e^{-\alpha_m} \sum_{i=1}^N p_{mi}$$

$$\text{最终令 } \frac{\partial L(y, F(x))}{\partial \alpha_m} = 0$$

$$\therefore \alpha_m^* = \frac{1}{2} \log \frac{1 - e_m}{e_m}$$

$$\text{其中, } e_m = \frac{\sum_{i=1}^N p_{mi} I(y_i \neq f_m(x))}{\sum_{i=1}^N p_{mi}} = \sum_{i=1}^N w_{mi} I(y_i \neq f_m(x)),$$

表示基础决策树 m 的错误率。

提升树之Adaboost算法的介绍

Adaboost分类算法步骤

- 在第一轮基础决策树 $f_1(x)$ 的构建中, 会设置每一个样本点的权重 w_{1i} 均为 $1/N$ 。
- 计算基础决策树 $f_m(x)$ 在训练数据集上的误判率 $e_m = \sum_{i=1}^N w_{mi} * I(y_i \neq f_m(x_i))$ 。
- 计算基础决策树 $f_m(x)$ 所对应的权重 $\alpha_m^* = \frac{1}{2} \log \frac{1-e_m}{e_m}$ 。
- 根据基础决策树 $f_m(x)$ 的预测结果, 计算下一轮用于构建基础决策树的样本点权重 $w_{m+1,i}^*$:

$$w_{m+1,i}^* = \begin{cases} \frac{w_{mi} \exp(-\alpha_m^*)}{\sum_{i=1}^N w_{mi} \exp(-\alpha_m^*)}, & f_m(x_i)^* = y_i \\ \frac{w_{mi} \exp(\alpha_m^*)}{\sum_{i=1}^N w_{mi} \exp(\alpha_m^*)}, & f_m(x_i)^* \neq y_i \end{cases}$$

提升树之Adaboost算法的介绍

Adaboost回归算法步骤

- ✦ 初始化一棵仅包含根节点的树，并寻找到一个常数能够使损失函数达到极小值。
- ✦ 计算第 m 轮基础树的残差值 $r_{mi} = y_i - f_m(x_i)$ 。
- ✦ 将残差值 r_{mi} 视为因变量，再利用自变量的值对其构造第 $m + 1$ 轮基础树 $f_{m+1}(x)$ 。
- ✦ 重复步骤（2）和（3），得到多棵基础树，最终将这些基础树相加得到回归提升树 $F(x) = \sum_{m=1}^M f_m(x)$ 。

提升树之Adaboost算法的应用

Adaboost函数语法

```
AdaBoostClassifier(base_estimator=None, n_estimators=50,  
                    learning_rate=1.0, algorithm='SAMME.R', random_state=None)
```

```
AdaBoostRegressor(base_estimator=None, n_estimators=50,  
                   learning_rate=1.0, loss='linear', random_state=None)
```

base_estimator: 用于指定提升算法所应用的基础分类器，默认为分类决策树（CART），也可以是其他基础分类器，但分类器必须支持带样本权重的学习，如神经网络。

n_estimators: 用于指定基础分类器的数量，默认为50个，当模型在训练数据集中得到完美的拟合后，可以提前结束算法，不一定非得构建完指定个数的基础分类器。

learning_rate: 用于指定模型迭代的学习率或步长，即对应的提升模型 $F(x)$ 可以表示为 $F(x) = F_{m-1}(x) + v\alpha_m f_m(x)$ ，其中的 v 就是该参数的指定值，默认值为1；对于较小的学习率 v 而言，则需要迭代更多次的基础分类器，通常情况下需要利用交叉验证法确定合理的基础分类器个数和学习率。

提升树之Adaboost算法的应用

Adaboost函数语法

```
AdaBoostClassifier(base_estimator=None, n_estimators=50,  
                    learning_rate=1.0, algorithm='SAMME.R', random_state=None)
```

```
AdaBoostRegressor(base_estimator=None, n_estimators=50,  
                   learning_rate=1.0, loss='linear', random_state=None)
```

algorithm: 用于指定AdaBoostClassifier分类器的算法，默认为'SAMME.R'，也可以使用'SAMME'；使用'SAMME.R'时，基础模型必须能够计算类别的概率值；一般而言，'SAMME.R'算法相比于'SAMME'算法，收敛更快、误差更小、迭代数量更少。

loss: 用于指定AdaBoostRegressor回归提升树的损失函数，可以是'linear'，表示使用线性损失函数；也可以是'square'，表示使用平方损失函数；还可以是'exponential'，表示使用指数损失函数；该参数的默认值为'linear'。

random_state: 用于指定随机数生成器的种子。

提升树之Adaboost算法的应用

信用卡是否违约的识别

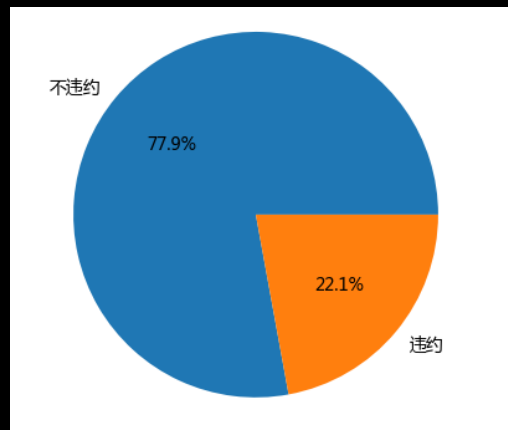
ID	LIMIT_B/	SEX	EDUCATI	MARRIAC	AGE	PAY_0	PAY_2	PAY_3	PAY_4	PAY_5	PAY_6	BILL_AM
1	20000	2	2	1	24	2	2	-1	-1	-2	-2	3913
2	120000	2	2	2	26	-1	2	0	0	0	0	2682
3	90000	2	2	2	34	0	0	0	0	0	0	29239
4	50000	2	2	1	37	0	0	0	0	0	0	46990
5	50000	1	2	1	57	-1	0	-1	0	0	0	8617
6	50000	1	1	2	37	0	0	0	0	0	0	64400
7	500000	1	1	2	29	0	0	0	0	0	0	367965
8	100000	2	2	2	23	0	-1	-1	0	0	-1	11876
9	140000	2	3	1	28	0	0	2	0	0	0	11285
10	20000	1	3	2	35	-2	-2	-2	-2	-1	-1	0
11	200000	2	3	2	34	0	0	2	0	0	-1	11073

BILL_AM	BILL_AM	BILL_AM	BILL_AM	BILL_AM	BILL_AM	PAY_AM	PAY_AM	PAY_AM	PAY_AM	PAY_AM	PAY_AM	y
3913	3102	689	0	0	0	0	689	0	0	0	0	1
2682	1725	2682	3272	3455	3261	0	1000	1000	1000	0	2000	1
29239	14027	13559	14331	14948	15549	1518	1500	1000	1000	1000	5000	0
46990	48233	49291	28314	28959	29547	2000	2019	1200	1100	1069	1000	0
8617	5670	35835	20940	19146	19131	2000	36681	10000	9000	689	679	0
64400	57069	57608	19394	19619	20024	2500	1815	657	1000	1000	800	0
367965	412023	445007	542653	483003	473944	55000	40000	38000	20239	13750	13770	0
11876	380	601	221	-159	567	380	601	0	581	1687	1542	0
11285	14096	12108	12211	11793	3719	3329	0	432	1000	1000	1000	0
0	0	0	0	13007	13912	0	0	0	13007	1122	0	0

提升树之Adaboost算法的应用

信用卡是否违约的识别

```
# 读入数据
default = pd.read_excel(r'default of credit card.xls')
# 为确保绘制的饼图为圆形，需执行如下代码
plt.axes(aspect = 'equal' )
# 中文乱码和坐标轴负号的处理
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False
# 统计客户是否违约的频数
counts = default.y.value_counts()
# 绘制饼图
plt.pie(x = counts, # 绘图数据
        labels=pd.Series(counts.index).map({0:'不违约',1:'违约'}),
        autopct='%.1f%%' # 设置百分比的格式，这里保留一位小数
    )
# 显示图形
plt.show()
```



提升树之Adaboost算法的应用

信用卡是否违约的识别

```
# 排除数据集中的ID变量和因变量, 剩余的数据用作自变量X
X = default.drop(['ID','y'], axis = 1)
y = default.y
# 数据拆分
X_train,X_test,y_train,y_test = model_selection.train_test_split(X,y,test_size = 0.25,
                                                                    random_state = 1234)

# 构建AdaBoost算法的类
AdaBoost1 = ensemble.AdaBoostClassifier()
# 算法在训练数据集上的拟合
AdaBoost1.fit(X_train,y_train)
# 算法在测试数据集上的预测
pred1 = AdaBoost1.predict(X_test)
```


提升树之Adaboost算法的应用

信用卡是否违约的识别

```
# 返回模型的预测效果
print('模型的准确率为: \n',metrics.accuracy_score(y_test, pred1))
print('模型的评估报告: \n',metrics.classification_report(y_test, pred1))
```

out:

模型的准确率为:
0.812533333333
模型的评估报告:

	precision	recall	f1-score	support
0	0.83	0.96	0.89	5800
1	0.68	0.32	0.44	1700
avg / total	0.80	0.81	0.79	7500

提升树之Adaboost算法的应用

信用卡是否违约的识别

```
# 计算客户违约的概率值, 用于生成ROC曲线的数据
y_score = AdaBoost1.predict_proba(X_test)[:,1]
fpr,tpr,threshold = metrics.roc_curve(y_test, y_score)

# 计算AUC的值
roc_auc = metrics.auc(fpr,tpr)

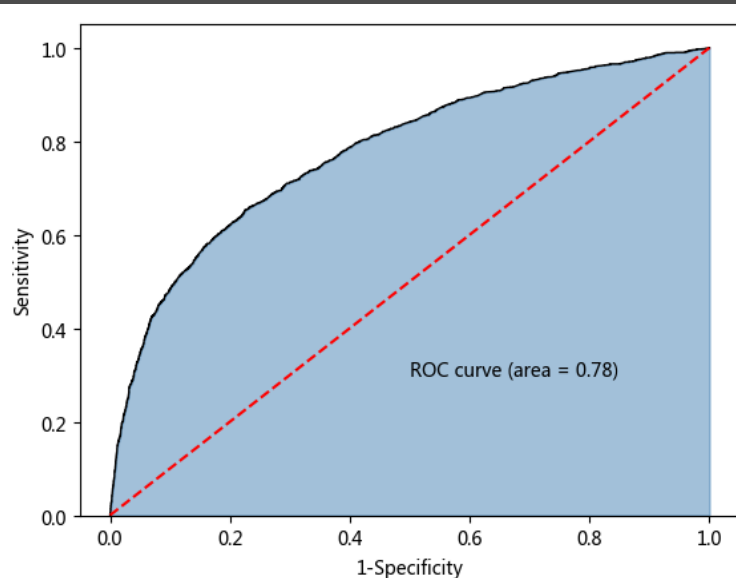
# 绘制面积图
plt.stackplot(fpr, tpr, color='steelblue', alpha = 0.5,
edgecolor = 'black')

# 添加边际线
plt.plot(fpr, tpr, color='black', lw = 1)

# 添加对角线
plt.plot([0,1],[0,1], color = 'red', linestyle = '--')

# 添加文本信息
plt.text(0.5,0.3,'ROC curve (area = %0.2f)' % roc_auc)
# 添加x轴与y轴标签
```

```
plt.xlabel('1-Specificity')
plt.ylabel('Sensitivity')
# 显示图形
plt.show()
```



提升树之Adaboost算法的应用

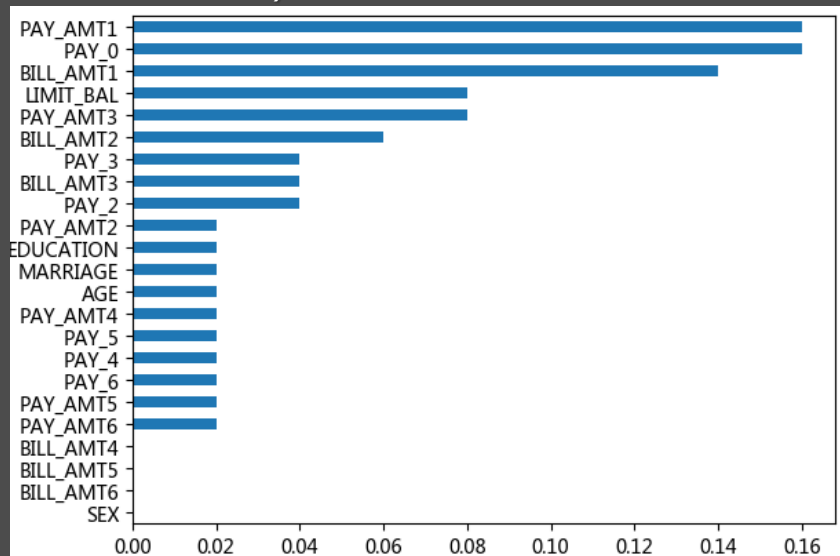
信用卡是否违约的识别

自变量的重要性排序

```
importance = pd.Series(AdaBoost1.feature_importances_, index = X.columns)
```

```
importance.sort_values().plot(kind = 'barh')
```

```
plt.show()
```



提升树之Adaboost算法的应用

信用卡是否违约的识别

```
# 取出重要性比较高的自变量建模
predictors = list(importance[importance>0.02].index)
Predictors
# 通过网格搜索法选择基础模型所对应的合理参数组合
max_depth = [3,4,5,6]
params1 = {'base_estimator__max_depth':max_depth}
base_model = GridSearchCV(estimator = ensemble.AdaBoostClassifier(base_estimator =
                                                                    DecisionTreeClassifier()),param_grid= params1,
                                                                    scoring = 'roc_auc', cv = 5, n_jobs = 4, verbose = 1)
base_model.fit(X_train[predictors],y_train)
# 返回参数的最佳组合和对应AUC值
base_model.best_params_, base_model.best_score_

out:
{'base_estimator__max_depth': 3}, 0.74425046797936145
```

提升树之Adaboost算法的应用

信用卡是否违约的识别

```
# 通过网格搜索法选择合理的Adaboost算法参数
n_estimators = [100,200,300]
learning_rate = [0.01,0.05,0.1,0.2]
params2 = {'n_estimators':n_estimators,'learning_rate':learning_rate}
adaboost = GridSearchCV(estimator = ensemble.AdaBoostClassifier(base_estimator =
                        DecisionTreeClassifier(max_depth = 3)),param_grid= params2,
                        scoring = 'roc_auc', cv = 5, n_jobs = 4, verbose = 1)
adaboost.fit(X_train[predictors], y_train)
# 返回参数的最佳组合和对应AUC值
adaboost.best_params_, adaboost.best_score_

out:
{'learning_rate': 0.01, 'n_estimators': 300}, 0.76866547085583281
```

提升树之Adaboost算法的应用

信用卡是否违约的识别

```
# 使用最佳的参数组合构建AdaBoost模型
```

```
AdaBoost2 = ensemble.AdaBoostClassifier(base_estimator = DecisionTreeClassifier(max_depth = 3),n_estimators = 300, learning_rate = 0.01)
```

```
# 算法在训练数据集上的拟合
```

```
AdaBoost2.fit(X_train[predictors],y_train)
```

```
# 算法在测试数据集上的预测
```

```
pred2 = AdaBoost2.predict(X_test[predictors])
```

提升树之Adaboost算法的应用

信用卡是否违约的识别

```
# 返回模型的预测效果
print('模型的准确率为: \n',metrics.accuracy_score(y_test, pred2))
print('模型的评估报告: \n',metrics.classification_report(y_test, pred2))
```

out:

模型的准确率为:
0.816

梯度提升树之GBDT算法的介绍

模型介绍

梯度提升树算法实际上是提升算法的扩展版，在原始的提升算法中，如果损失函数为平方损失或指数损失，求解损失函数的最小值问题会非常简单，但如果损失函数为更一般的函数（如绝对值损失函数或Huber损失函数等），目标值的求解就会相对复杂很多。

梯度提升算法，是在第 m 轮基础模型中，利用损失函数的负梯度值作为该轮基础模型损失值的近似，并利用这个近似值构建下一轮基础模型。利用损失函数的负梯度值近似残差的计算就是梯度提升算法在提升算法上的扩展，这样的扩展使得目标函数的求解更为方便。

梯度提升树之GBDT算法的介绍

GBDT算法步骤

- ✦ 初始化一棵仅包含根节点的树，并寻找到一个常数Const能够使损失函数达到极小值
- ✦ 计算损失函数的负梯度值，用作残差的估计值，即：

$$r_{mi} = - \left[\frac{\partial L(y_i, f(x_i))}{\partial f(x_i)} \right]_{f(x)=f_{m-1}(x)}$$

- ✦ 利用数据集 (x_i, r_{mi}) 拟合下一轮基础模型，得到对应的 J 个叶子节点 R_{mj} , $j = 1, 2, \dots, J$; 计算每个叶子节点 R_{mj} 的最佳拟合值，用以估计残差 r_{mi} ：

$$c_{mj} = \operatorname{argmin}_c \sum_{x_i \in R_{mj}} L(y_i, f_{m-1}(x_i) + c)$$

梯度提升树之GBDT算法的介绍

GBDT算法步骤



进而得到第 m 轮的基础模型 $f_m(x)$ ，再结合前 $m - 1$ 轮的基础模型，得到最终的梯度提升模型：

$$\begin{aligned} F_M(x) &= F_{M-1}(x) + f_m(x) \\ &= F_{M-1}(x) + \sum_{j=1}^J c_{mj} I(x_i \in R_{mj}) \\ &= \sum_{m=1}^M \sum_{j=1}^J c_{mj} I(x_i \in R_{mj}) \end{aligned}$$

其中， c_{mj} 表示第 m 个基础模型 $f_m(x)$ 在叶节点 j 上的预测值； $F_M(x)$ 表示由 M 个基础模型构成的梯度提升树，它是每一个基础模型在样本点 x_i 处的输出值 c_{mj} 之和。

梯度提升树之GBDT算法的介绍

GBDT算法在分类问题上的操作步骤

✦ 初始化一个弱分类器

$$f_0(x) = \operatorname{argmin}_c \sum_{i=1}^n L(y_i, c)$$

✦ 计算损失函数的负梯度值

$$\begin{aligned} r_{mi} &= - \left[\frac{\partial L(y_i, f(x_i))}{\partial f(x_i)} \right]_{f(x)=f_{m-1}(x)} \\ &= - \left[\frac{\partial \log(1 + \exp(-y_i f(x_i)))}{\partial f(x_i)} \right]_{f(x)=f_{m-1}(x)} \\ &= \frac{y_i}{1 + \exp(-y_i f(x_i))} \end{aligned}$$

梯度提升树之GBDT算法的介绍

GBDT算法在分类问题上的操作步骤

✦ 利用数据集 (x_i, r_{mi}) 拟合下一轮基础模型

$$f_m(x) = \sum_{j=1}^J c_{mj} I(x_i \in R_{mj})$$

$$\text{其中, } c_{mj} = \operatorname{argmin}_c \sum_{x_i \in R_{mj}} \log(1 + \exp(-y_i(f_{m-1}(x_i) + c)))$$

✦ 重复 (2) 和 (3) , 并利用 m 个基础模型, 构建梯度提升模型

$$\begin{aligned} F_M(x) &= F_{M-1}(x) + f_m(x) \\ &= \sum_{m=1}^M \sum_{j=1}^J c_{mj} I(x_i \in R_{mj}) \end{aligned}$$

梯度提升树之GBDT算法的介绍

GBDT算法在预测问题上的操作步骤

✦ 初始化一个弱分类器

$$f_0(x) = \operatorname{argmin}_c \sum_{i=1}^n L(y_i, c)$$

✦ 计算损失函数的负梯度值

$$\begin{aligned} r_{mi} &= - \left[\frac{\partial L(y_i, f(x_i))}{\partial f(x_i)} \right]_{f(x)=f_{m-1}(x)} \\ &= - \left[\frac{\partial \frac{1}{2} (y_i - f(x_i))^2}{\partial f(x_i)} \right]_{f(x)=f_{m-1}(x)} \\ &= y_i - f(x_i) \end{aligned}$$

梯度提升树之GBDT算法的介绍

GBDT算法在预测问题上的操作步骤

- ★ 利用数据集 (x_i, r_{mi}) 拟合下一轮基础模型

$$f_m(x) = \sum_{j=1}^J c_{mj} I(x_i \in R_{mj})$$

其中, $c_{mj} = \operatorname{argmin}_c \sum_{x_i \in R_{mj}} \frac{1}{2} (y_i - (f_{m-1}(x_i) + c))^2$ 。

- ★ 重复 (2) 和 (3) , 并利用 m 个基础模型, 构建梯度提升模型

$$F_M(x) = F_{M-1}(x) + f_m(x) = \sum_{m=1}^M \sum_{j=1}^J c_{mj} I(x_i \in R_{mj})$$

梯度提升树之GBDT算法的应用

GBDT函数介绍

GradientBoostingClassifier()

GradientBoostingRegressor(loss='ls', learning_rate=0.1, n_estimators=100, subsample=1.0, criterion='friedman_mse', min_samples_split=2, min_samples_leaf=1, min_weight_fraction_leaf=0.0, max_depth=3, min_impurity_decrease=0.0, min_impurity_split=None, init=None, random_state=None, max_features=None, alpha=0.9, verbose=0, max_leaf_nodes=None, warm_start=False, presort='auto')

loss: 用于指定GBDT算法的损失函数，对于分类的GBDT，可以选择'deviance'和'exponential'，分别表示对数似然损失函数和指数损失函数；对于预测的GBDT，可以选择'ls' 'lad' 'huber'和'quantile'，分别表示平方损失函数、绝对值损失函数、Huber损失函数（前两种损失函数的结合，当误差较小时，使用平方损失，否则使用绝对值损失，误差大小的度量可使用alpha参数指定）和分位数回归损失函数（需通过alpha参数设定分位数）。

learning_rate: 用于指定模型迭代的学习率或步长，即对应的梯度提升模型 $F(x)$ 可以表示为 $F_M(x) = F_{M-1}(x) + \nu f_m(x)$ ，其中的 ν 就是该参数的指定值，默认值为0.1；对于较小的学习率 ν 而言，则需要迭代更多次的基础分类器，通常情况下需要利用交叉验证法确定合理的基础模型的个数和学习率。

梯度提升树之GBDT算法的应用

GBDT函数介绍

```
GradientBoostingRegressor(loss='ls', learning_rate=0.1, n_estimators=100, subsample=1.0,  
                           criterion='friedman_mse', min_samples_split=2,  
                           min_samples_leaf=1, min_weight_fraction_leaf=0.0, max_depth=3,  
                           min_impurity_decrease=0.0, min_impurity_split=None, init=None,  
                           random_state=None, max_features=None, alpha=0.9, verbose=0,  
                           max_leaf_nodes=None, warm_start=False, presort='auto' )
```

n_estimators: 用于指定基础模型的数量，默认为100个。

subsample: 用于指定构建基础模型所使用的抽样比例，默认为1，表示使用原始数据构建每一个基础模型；当抽样比例小于1时，表示构建随机梯度提升树模型，通常会导致模型的方差降低，偏差提高。

criterion: 用于指定分割质量的度量，默认为'friedman_mse'，表示使用Friedman均方误差，还可以使用'mse'和'mae'，分别表示均方误差和绝对误差。

min_samples_split: 用于指定每个基础模型的根节点或中间节点能够继续分割的最小样本量，默认为2。

min_samples_leaf: 用于指定每个基础模型的叶节点所包含的最小样本量，默认为1。

min_weight_fraction_leaf: 用于指定每个基础模型叶节点最小的样本权重，默认为0，表示不考虑叶节点的样本权重。

梯度提升树之GBDT算法的应用

GBDT函数介绍

```
GradientBoostingRegressor(loss='ls', learning_rate=0.1, n_estimators=100, subsample=1.0,  
                           criterion='friedman_mse', min_samples_split=2,  
                           min_samples_leaf=1, min_weight_fraction_leaf=0.0, max_depth=3,  
                           min_impurity_decrease=0.0, min_impurity_split=None, init=None,  
                           random_state=None, max_features=None, alpha=0.9, verbose=0,  
                           max_leaf_nodes=None, warm_start=False, presort='auto' )
```

max_depth: 用于指定每个基础模型所包含的最大深度，默认为3层。

min_impurity_decrease: 用于指定每个基础模型的节点是否继续分割的最小不纯度值，默认为0；如果不纯度超过指定的阈值，则节点需要分割，否则不分割。

min_impurity_split: 该参数同min_impurity_decrease参数，在sklearn的0.21版本及之后版本将删除。

init: 用于指定初始的基础模型，用于执行初始的分类或预测。

random_state: 用于指定随机数生成器的种子，默认为None，表示使用默认的随机数生成器。

verbose: 用于指定GBDT算法在计算过程中是否输出日志信息，默认为0，表示不输出。

alpha: 当loss参数为'huber'或'quantile'时，该参数有效，分别用于指定误差的阈值和分位数。

梯度提升树之GBDT算法的应用

GBDT函数介绍

```
GradientBoostingRegressor(loss='ls', learning_rate=0.1, n_estimators=100, subsample=1.0,  
                           criterion='friedman_mse', min_samples_split=2,  
                           min_samples_leaf=1, min_weight_fraction_leaf=0.0, max_depth=3,  
                           min_impurity_decrease=0.0, min_impurity_split=None, init=None,  
                           random_state=None, max_features=None, alpha=0.9, verbose=0,  
                           max_leaf_nodes=None, warm_start=False, presort='auto' )
```

max_features: 用于指定每个基础模型所包含的最多分割字段数，默认为None，表示分割时使用所有的字段；如果为具体的整数，则考虑使用对应的分割字段数；如果为0~1的浮点数，则考虑对应百分比的字段个数；如果为'sqrt'，则表示最多考虑 $\sqrt{P}$ 个字段，与指定'auto'效果一致；如果为'log2'，则表示最多使用 $\log_2 P$ 个字段。其中， P 表示数据集所有自变量的个数。

max_leaf_nodes: 用于指定每个基础模型最大的叶节点个数，默认为None，表示对叶节点个数不做任何限制。

warm_start: bool类型参数，表示是否使用上一轮的训练结果，默认为False。

梯度提升树之GBDT算法的应用

信用卡是否违约的识别

```
# 运用网格搜索法选择梯度提升树的合理参数组合
learning_rate = [0.01,0.05,0.1,0.2]
n_estimators = [100,200,300]
max_depth = [3,4,5,6]
params = {'learning_rate':learning_rate,'n_estimators':n_estimators,'max_depth':max_depth}
gbdt_grid = GridSearchCV(estimator = ensemble.GradientBoostingClassifier(),
                        param_grid= params, scoring = 'roc_auc', cv = 5, n_jobs = 4, verbose = 1)
gbdt_grid.fit(X_train[predictors],y_train)
# 返回参数的最佳组合和对应AUC值
gbdt_grid.best_params_, gbdt_grid.best_score_
```

out:

```
{'learning_rate': 0.05, 'max_depth': 5, 'n_estimators': 100}, 0.77397780446802755
```

梯度提升树之GBDT算法的应用

信用卡是否违约的识别

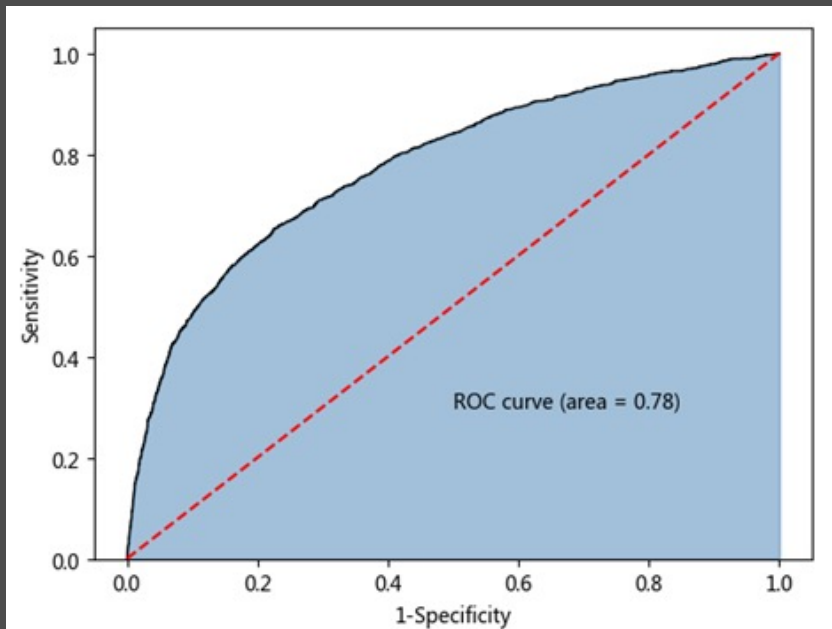
```
# 基于最佳参数值训练模型
gbdt = ensemble.GradientBoostingClassifier(learning_rate=0.05,n_estimators=100,max_depth=5)
gbdt.fit(X_train[predictors],y_train)
# 基于最佳参数组合的GBDT模型，对测试数据集进行预测
pred = gbdt.predict(X_test[predictors]) # 返回模型的预测效果
print('模型的准确率为：\n',metrics.accuracy_score(y_test, pred))
print('模型的评估报告：\n',metrics.classification_report(y_test, pred))
```

out:

模型的准确率为：
0.814266666667

梯度提升树之GBDT算法的应用

信用卡是否违约的识别



GBDT模型在测试数据集上的预测效果与AdaBoost算法基本一致，进而可以说说明GBDT算法采用一阶导函数的值近似残差是合理的，并且这种近似功能也提升了AdaBoost算法求解目标函数时的便捷性。

非平衡数据的处理

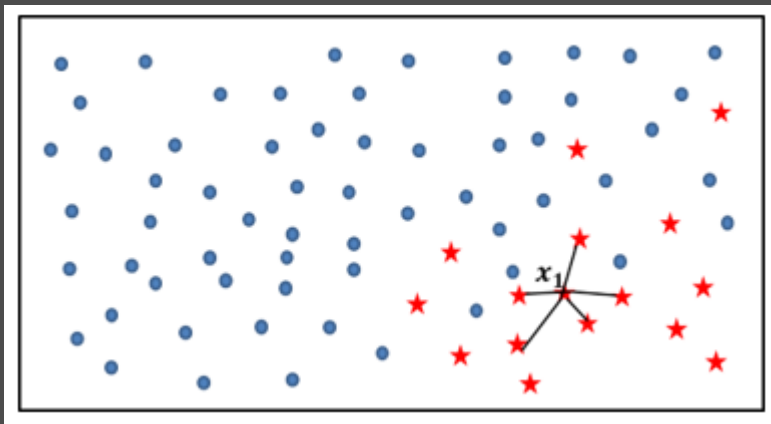
非平衡数据的特征

在实际应用中，类别型的因变量可能存在严重的偏倚，即类别之间的比例严重失调。如欺诈问题中，欺诈类观测在样本集中毕竟占少数；客户流失问题中，忠实的客户往往也是占很少一部分；在某营销活动的响应问题中，真正参与活动的客户也同样只是少部分。

如果数据存在严重的不平衡，预测得出的结论往往也是有偏的，即分类结果会偏向于较多观测的类。为了解决数据的非平衡问题，2002年Chawla提出了SMOTE算法，即合成少数过采样技术，它是基于随机过采样算法的一种改进方案。

非平衡数据的处理

SMOTE算法的思想



SMOTE算法的基本思想就是对少数类别样本进行分析和模拟，并将人工模拟的新样本添加到数据集中，进而使原始数据中的类别不再严重失衡。

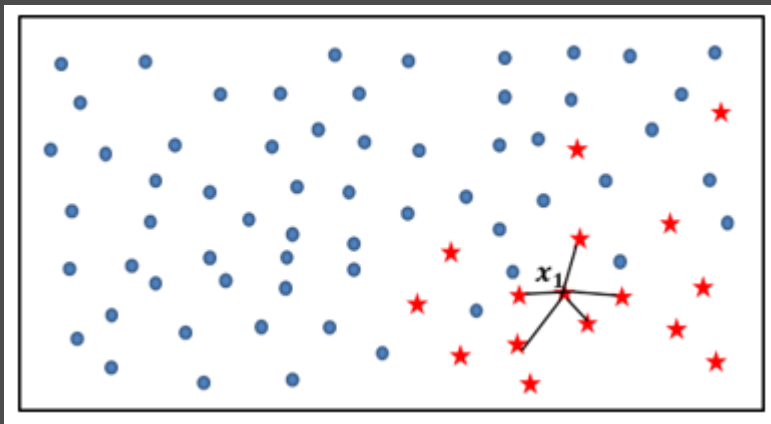
非平衡数据的处理

SMOTE算法的步骤

- ✦ 采样最邻近算法，计算出每个少数类样本的K个近邻。
- ✦ 从K个近邻中随机挑选N个样本进行随机线性插值。
- ✦ 构造新的少数类样本。
- ✦ 将新样本与原数据合成，产生新的训练集。

非平衡数据的处理

SMOTE算法的手工案例

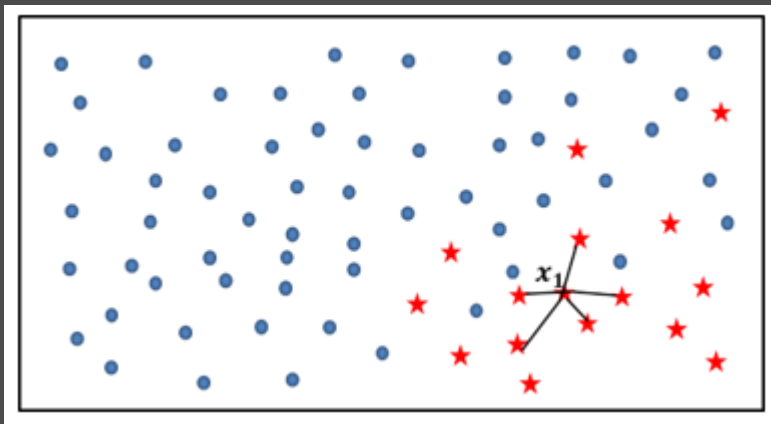


(1) 利用KNN算法，选择离样本点 x_1 最近的K个同类样本点（不妨最近邻为5）。

(2) 从最近的K个同类样本点中，随机挑选M个样本点（不妨设M为2），M的选择依赖于最终所希望的平衡率。

非平衡数据的处理

SMOTE算法的手工案例



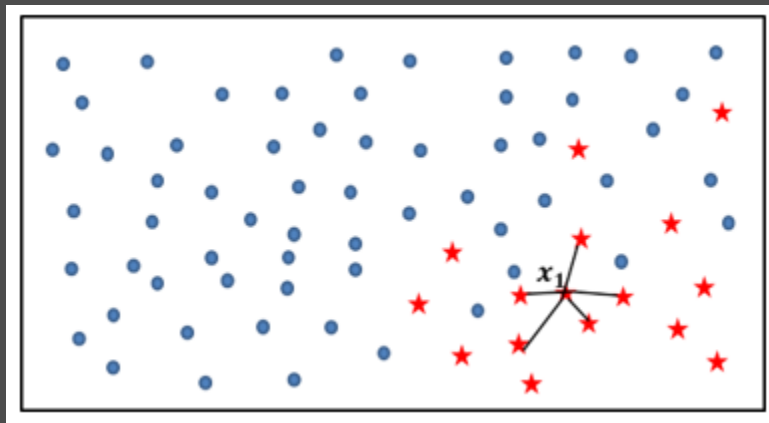
(3) 对于每一个随机选中的样本点，构造新的样本点。新样本点的构造需要使用下方的公式：

$$x_{new} = x_i + rand(0,1) \times (x_j - x_i), \quad j = 1, 2, \dots, M$$

其中， x_i 表示少数类别中的一个样本点（如图中五角星所代表的 x_1 样本）； x_j 表示从K近邻中随机挑选的样本点 j ； $rand(0,1)$ 表示生成0~1的随机数。

非平衡数据的处理

SMOTE算法的手工案例



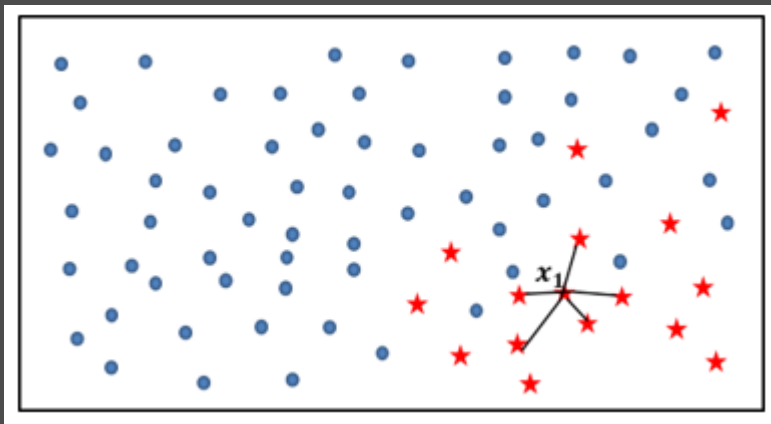
假设图中样本点 x_1 的观测值为(2,3,10,7)，从图中的5个近邻随机挑选两个样本点，它们的观测值分别为(1,1,5,8)和(2,1,7,6)，由此得到的两个新样本点为：

$$x_{new1} = (2,3,10,7) + 0.3 \times ((1,1,5,8) - (2,3,10,7)) = (1.7, 2.4, 8.5, 7.3)$$

$$x_{new2} = (2,3,10,7) + 0.26 \times ((2,1,7,6) - (2,3,10,7)) = (2, 2.48, 9.22, 6.74)$$

非平衡数据的处理

SMOTE算法的手工案例



(4) 重复步骤 (1)、(2) 和 (3)，通过迭代少数类别中的每一个样本 x_i ，最终将原始的少数类别样本量扩大为理想的比例。

imbalanced-learn documentation

[✎ Edit on GitHub](#)[📄 Show Source](#)

Date: May 28, 2024 **Version:** 0.12.3

Useful links: [Binary Installers](#) | [Source Repository](#) | [Issues & Ideas](#) | [Q&A Support](#)

Imbalanced-learn (imported as `imblearn`) is an open source, MIT-licensed library relying on scikit-learn (imported as `sklearn`) and provides tools when dealing with classification with imbalanced classes.



Getting started

Check out the getting started guides to install `imbalanced-learn`. Some extra information to get started with a new contribution is also provided.

To the installation
guideline



User guide

The user guide provides in-depth information on the key concepts of `imbalanced-learn` with useful background information and explanation.

To the user guide

非平衡数据的处理

SMOTE算法的函数介绍

```
from imblearn.over_sampling import SMOTE
```

```
SMOTE(ratio='auto', random_state=None, k_neighbors=5, m_neighbors=10,  
      out_step=0.5, kind='regular', svm_estimator=None, n_jobs=1)
```

ratio: 用于指定重抽样的比例，如果指定字符型的值，可以是'minority'（表示对少数类别的样本进行抽样）、'majority'（表示对多数类别的样本进行抽样）、'not minority'（表示采用欠采样方法）、'all'（表示采用过采样方法），默认为'auto'，等同于'all'和'not minority'。如果指定字典型的值，其中键为各个类别标签，值为类别下的样本量。

random_state: 用于指定随机数生成器的种子，默认为None，表示使用默认的随机数生成器。

k_neighbors: 指定近邻个数，默认为5个。

m_neighbors: 指定从近邻样本中随机挑选的样本个数，默认为10个。

kind: 用于指定SMOTE算法在生成新样本时所使用的选项，默认为'regular'，表示对少数类别的样本进行随机采样，也可以是'borderline1' 'borderline2'和'svm'。

svm_estimator: 用于指定SVM分类器，默认为sklearn.svm.SVC，该参数的目的是利用支持向量机分类器生成支持向量，然后生成新的少数类别的样本。

n_jobs: 用于指定SMOTE算法在过采样时所需的CPU数量，默认为1表示仅使用1个CPU运行算法，即不使用并行运算功能。

DBDT的改进之XGBoost算法介绍

XGBoost算法的介绍

XGBoost是由传统的GBDT模型发展而来的，GBDT模型在求解最优化问题时应用了一阶导技术，而XGBoost则使用损失函数的一阶和二阶导，而且可以自定义损失函数，只要损失函数可一阶和二阶求导。

XGBoost算法相比于GBDT算法还有其他优点，例如支持并行计算，大大提高算法的运行效率；XGBoost在损失函数中加入了正则项，用来控制模型的复杂度，进而可以防止模型的过拟合；XGBoost除了支持CART基础模型，还支持线性基础模型；XGBoost采用了随机森林的思想，对字段进行抽样，既可以防止过拟合，也可以降低模型的计算量。

DBDT的改进之XGBoost算法介绍

XGBoost算法的损失函数

$$\hat{y}_i^{(t)} = \sum_{k=1}^t f_k(x_i) = \hat{y}_i^{(t-1)} + f_t(x_i)$$

其中, $\hat{y}_i^{(t)}$ 表示经第 t 轮迭代后的模型预测值, $\hat{y}_i^{(t-1)}$ 表示已知 $t - 1$ 个基础模型的预测值, $f_t(x_i)$ 表示第 t 个基础模型。

对于集成树, 关键点就是第 t 个基础模型 f_t 的选择。所以, 只需要寻找一个能够使目标函数尽可能最大化降低的 f_t 即可, 故构造的目标函数如下:

$$\begin{aligned} Obj^{(t)} &= \sum_{i=1}^n L(y_i, \hat{y}_i^{(t)}) + \sum_{j=1}^t \Omega(f_j) \\ &= \sum_{i=1}^n L(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)) + \sum_{j=1}^t \Omega(f_j) \end{aligned}$$

DBDT的改进之XGBoost算法介绍

XGBoost算法的损失函数

损失函数中的 $\Omega(f_j)$ 为第 j 个基础模型的正则项，用于控制模型的复杂度。为了简单起见，不妨将损失函数 L 表示为平方损失，则如上的目标函数可以表示为：

$$\begin{aligned} Obj^{(t)} &= \sum_{i=1}^n \left(y_i - \left(\hat{y}_i^{(t-1)} + f_t(x_i) \right) \right)^2 + \sum_{j=1}^t \Omega(f_j) \\ &= \sum_{i=1}^n \left(y_i^2 + \left(\hat{y}_i^{(t-1)} + f_t(x_i) \right)^2 - 2y_i \left(\hat{y}_i^{(t-1)} + f_t(x_i) \right) \right) + \sum_{j=1}^t \Omega(f_j) \\ &= \sum_{i=1}^n \left(2f_t(x_i) \left(\hat{y}_i^{(t-1)} - y_i \right) + f_t(x_i)^2 + \left(y_i - \hat{y}_i^{(t-1)} \right)^2 \right) + \sum_{j=1}^t \Omega(f_j) \end{aligned}$$

DBDT的改进之XGBoost算法介绍

XGBoost算法的损失函数

由于前 $t - 1$ 个基础模型是已知的，故 $\hat{y}_i^{(t-1)}$ 的预测值也是已知的，同时前 $t - 1$ 个基础模型的复杂度也是已知的，故不妨将所有的已知项设为常数 *constant*，则目标函数可以重新表达为：

$$Obj^{(t)} = \sum_{i=1}^n \left(2f_t(x_i) \left(\hat{y}_i^{(t-1)} - y_i \right) + f_t(x_i)^2 \right) + \Omega(f_t) + constant$$

其中， $\left(\hat{y}_i^{(t-1)} - y_i \right)$ 项就是前 $t - 1$ 个基础模型所产生的残差，说明目标函数的选择与前 $t - 1$ 个基础模型的残差相关，这一点与GBDT是相同的。如上假设损失函数为平方损失，对于更一般的损失函数来说，可以使用泰勒展开对损失函数值做近似估计。

DBDT的改进之XGBoost算法介绍

泰勒展开式

$$f(x + \Delta x) \approx f(x) + f(x)' \Delta x + f(x)'' \Delta x^2$$

其中, $f(x)$ 是一个具有二阶可导的函数, $f(x)'$ 为 $f(x)$ 的一阶导函数, $f(x)''$ 为 $f(x)$ 的二阶导函数, Δx 为 $f(x)$ 在某点处的变化量。假设令损失函数 L 为泰勒公式中的 f , 令损失函数中 $\hat{y}_i^{(t-1)}$ 项为泰勒公式中的 x , 令损失函数中 $f_t(x_i)$ 项为泰勒公式中的 Δx , 则目标函数 $Obj^{(t)}$ 可以近似表示为:

$$\begin{aligned} Obj^{(t)} &= \sum_{i=1}^n L\left(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)\right) + \Omega(f_t) + constant \\ &\approx \sum_{i=1}^n \left(L\left(y_i, \hat{y}_i^{(t-1)}\right) + g_i f_t(x_i) + \frac{1}{2} h_i f_t(x_i)^2 \right) + \Omega(f_t) + constant \end{aligned}$$

DBDT的改进之XGBoost算法介绍

泰勒展开式

在上式中, g_i 和 h_i 分别是损失函数 $L(y_i, \hat{y}_i^{(t-1)})$ 关于 $\hat{y}_i^{(t-1)}$ 的一阶导函数值和二阶导函数值, 即它们可以表示为:

$$\begin{cases} g_i = \frac{\partial L(y_i, \hat{y}_i^{(t-1)})}{\partial \hat{y}_i^{(t-1)}} \\ h_i = \frac{\partial^2 L(y_i, \hat{y}_i^{(t-1)})}{\partial \hat{y}_i^{(t-1)}} \end{cases}$$

DBDT的改进之XGBoost算法介绍

损失函数的演变

假设基础模型 f_t 由CART树构成，对于一棵树来说，它可以被拆分为结构部分 q ，以及叶子节点所对应的输出值 w 。可以利用这两部分反映树的复杂度，即复杂度由树的叶子节点个数（反映树的结构）和叶子节点输出值的平方构成：

$$\Omega(f_t) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2$$

其中， T 表示叶子节点的个数， w_j^2 表示输出值向量的平方。CART树生长得越复杂，对应的 T 越大， $\Omega(f_t)$ 也越大。

DBDT的改进之XGBoost算法介绍

损失函数的演变

$$Obj^{(t)} \approx \sum_{i=1}^n \left(L(y_i, \hat{y}_i^{(t-1)}) + g_i f_t(x_i) + \frac{1}{2} h_i f_t(x_i)^2 \right) + \Omega(f_t) + constant$$

$$\approx \sum_{i=1}^n \left(g_i f_t(x_i) + \frac{1}{2} h_i f_t(x_i)^2 \right) + \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2 + constant$$

$$\approx \sum_{i=1}^n \left(g_i w_{q(x_i)} + \frac{1}{2} h_i w_{q(x_i)}^2 \right) + \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2 + constant$$

$$\approx \sum_{j=1}^T \left(\left(\sum_{i \in I_j} g_i \right) w_j + \frac{1}{2} \left(\sum_{i \in I_j} h_i \right) w_j^2 \right) + \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2 + constant$$

$$\approx \sum_{j=1}^T \left(\left(\sum_{i \in I_j} g_i \right) w_j + \frac{1}{2} \left(\sum_{i \in I_j} (h_i + \lambda) \right) w_j^2 \right) + \gamma T + constant$$

DBDT的改进之XGBoost算法介绍

损失函数的演变

$$\begin{aligned} Obj^{(t)} &\approx \sum_{j=1}^T \left(\left(\sum_{i \in I_j} g_i \right) w_j + \frac{1}{2} \left(\sum_{i \in I_j} (h_i + \lambda) \right) w_j^2 \right) + \gamma T + constant \\ &\approx \sum_{j=1}^T \left(\left(\sum_{i \in I_j} g_i \right) w_j + \frac{1}{2} \left(\sum_{i \in I_j} (h_i + \lambda) \right) w_j^2 \right) + \gamma T \\ &\approx \sum_{j=1}^T \left(G_j w_j + \frac{1}{2} (H_j + \lambda) w_j^2 \right) + \gamma T \end{aligned}$$

其中, $G_j = \sum_{i \in I_j} g_i$; $H_j = \sum_{i \in I_j} h_i$ 。它们分别表示所有属于叶子节点 j 的样本点对应的 g_i 之和以及 h_i 之和。所以, 最终是寻找一个合理的 f_t , 使得式子 $\sum_{j=1}^T \left(G_j w_j + \frac{1}{2} (H_j + \lambda) w_j^2 \right) + \gamma T$ 尽可能大地减小。

DBDT的改进之XGBoost算法介绍

损失函数的演变

✦ 求偏导，令导函数为0

$$\frac{\partial Obj^{(t)}}{\partial w_j} = G_j + (H_j + \lambda)w_j = 0$$

$$\therefore w_j = -\frac{G_j}{H_j + \lambda}$$

所以，将 w_j 的值导入到目标函数 $Obj^{(t)}$ 中，可得：

$$\begin{aligned} J(f_t) &= \sum_{j=1}^T \left(G_j w_j + \frac{1}{2} (H_j + \lambda) w_j^2 \right) + \gamma T \\ &= -\frac{1}{2} \sum_{j=1}^T \left(\frac{G_j^2}{H_j + \lambda} \right) + \gamma T \end{aligned}$$

DBDT的改进之XGBoost算法介绍

损失函数的演变

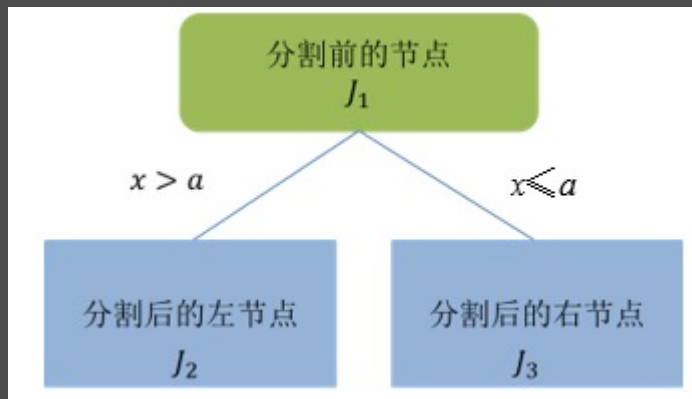
为了能够寻找到最佳的树结构 q (即最佳的基础模型 f_t) , 通常会选择贪心法, 即在某个已有的可划分节点中加入一个分割, 并通过计算分割前后的增益值决定是否剪枝。有关增益值的计算如下:

$$Gain = \frac{1}{2} \left(\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda} \right) - \gamma$$

其中, G_L 和 H_L 为某节点分割后对应的左支样本点的导函数值, G_R 和 H_R 为某节点分割后对应的右支样本点的导函数值。这里的增益值 $Gain$ 其实就是将某节点分割为另外两个节点后对应的目标值 $J(f_t)$ 的减少量。

DBDT的改进之XGBoost算法介绍

损失函数的演变



其中, J_1 表示某个可分割节点在分割前的目标函数值, J_2 和 J_3 则代表该节点按照某个变量 x 在 a 处的分割后对应的目标函数值。按照目标函数的公式, 可以将这三个值表示为下方的式子:

$$\begin{cases} J_1 = -\frac{1}{2} \left(\frac{(G_L + G_R)^2}{H_L + H_R + \lambda} \right) + \gamma \\ J_2 = -\frac{1}{2} \left(\frac{G_L^2}{H_L + \lambda} \right) + \gamma \\ J_3 = -\frac{1}{2} \left(\frac{G_R^2}{H_R + \lambda} \right) + \gamma \end{cases}$$

XGBoost算法的应用实战

XGBoost函数介绍

```
XGBClassifier(max_depth=3, learning_rate=0.1, n_estimators=100, silent=True,  
               objective='binary:logistic', booster='gbtree', n_jobs=1, nthread=None,  
               gamma=0, min_child_weight=1, max_delta_step=0, subsample=1,  
               colsample_bytree=1, colsample_bylevel=1, reg_alpha=0, reg_lambda=1,  
               scale_pos_weight=1, base_score=0.5, random_state=0, seed=None, missing=None)
```

max_depth: 用于指定每个基础模型所包含的最大深度，默认为3层。

learning_rate: 用于指定模型迭代的学习率或步长，默认为0.1，即对应的梯度提升模型 $F_T(x)$ 可以表示为 $F_T(x) = F_{T-1}(x) + \eta f_t(x)$ ，其中的 η 就是该参数的指定值，默认值为1；对于较小的学习率 η 而言，则需要迭代更多次的基础分类器，通常情况下需要利用交叉验证法确定合理的基础模型的个数和学习率。

n_estimators: 用于指定基础模型的数量，默认为100个。

silent: bool类型参数，是否输出算法运行过程中的日志信息，默认为True。

booster: 用于指定基础模型的类型，默认为'gbtree'，即CART模型，也可以是'gblinear'，表示基础模型为线性模型。

XGBoost算法的应用实战

XGBoost函数介绍

```
XGBClassifier(max_depth=3, learning_rate=0.1, n_estimators=100, silent=True,  
              objective='binary:logistic', booster='gbtree', n_jobs=1, nthread=None,  
              gamma=0, min_child_weight=1, max_delta_step=0, subsample=1,  
              colsample_bytree=1, colsample_bylevel=1, reg_alpha=0, reg_lambda=1,  
              scale_pos_weight=1, base_score=0.5, random_state=0, seed=None, missing=None)
```

objective: 用于指定目标函数中的损失函数类型，对于分类型的XGBoost算法，默认的损失函数为二分类的Logistic损失（模型返回概率值），也可以是'multi:softmax',表示用于处理多分类的损失函数（模型返回类别值），还可以是'multi:softprob', 与'multi:softmax'相同，所不同的是模型返回各类别对应的概率值；对于预测型的XGBoost算法，默认的损失函数为线性回归损失。

n_jobs: 用于指定XGBoost算法在并行计算时所需的CPU数量，默认为1表示仅使用1个CPU运行算法，即不使用并行运算功能。

nthread: 用于指定XGBoost算法在运行时所使用的线程数，默认为None，表示使用计算机最大可能的线程数。

gamma: 用于指定节点分割所需的最小损失函数下降值，即增益值Gain的阈值，默认为0。

XGBoost算法的应用实战

XGBoost函数介绍

```
XGBClassifier(max_depth=3, learning_rate=0.1, n_estimators=100, silent=True,  
               objective='binary:logistic', booster='gbtree', n_jobs=1, nthread=None,  
               gamma=0, min_child_weight=1, max_delta_step=0, subsample=1,  
               colsample_bytree=1, colsample_bylevel=1, reg_alpha=0, reg_lambda=1,  
               scale_pos_weight=1, base_score=0.5, random_state=0, seed=None, missing=None)
```

min_child_weight: 用于指定叶子节点中各样本点二阶导之和的最小值，即 H_j 的最小值，默认为1，该参数的值越小，模型越容易过拟合。

max_delta_step: 用于指定模型在更新过程中的步长，如果为0，表示没有约束；如果取值为某个较小的正数，就会导致模型更加保守。

subsample: 用于指定构建基础模型所使用的抽样比例，默认为1，表示使用原始数据构建每一个基础模型；当抽样比例小于1时，表示构建随机梯度提升树模型，通常会导致模型的方差降低，偏差提高。

colsample_bytree: 用于指定每个基础模型所需的采样字段比例，默认为1，表示使用原始数据的所有字段。

colsample_bylevel: 用于指定每个基础模型在节点分割时所需的采样字段比例，默认为1，表示使用原始数据的所有字段。

XGBoost算法的应用实战

XGBoost函数介绍

```
XGBClassifier(max_depth=3, learning_rate=0.1, n_estimators=100, silent=True,  
               objective='binary:logistic', booster='gbtree', n_jobs=1, nthread=None,  
               gamma=0, min_child_weight=1, max_delta_step=0, subsample=1,  
               colsample_bytree=1, colsample_bylevel=1, reg_alpha=0, reg_lambda=1,  
               scale_pos_weight=1, base_score=0.5, random_state=0, seed=None, missing=None)
```

reg_alpha: 用于指定L1正则项的系数，默认为0。

reg_lambda: 用于指定L2正则项的系数，默认为1。

scale_pos_weight: 当各类别样本的比例十分不平衡时，通过设定该参数设定为一个正值，可以使算法更快收敛。

base_score: 用于指定所有样本的初始化预测得分，默认为0.5。

random_state: 用于指定随机数生成器的种子，默认为0，表示使用默认的随机数生成器。

seed: 同random_state参数。

missing: 用于指定缺失值的表示方法，默认为None，表示NaN即为默认值。

算法的应用实战

信用卡欺诈行为的识别

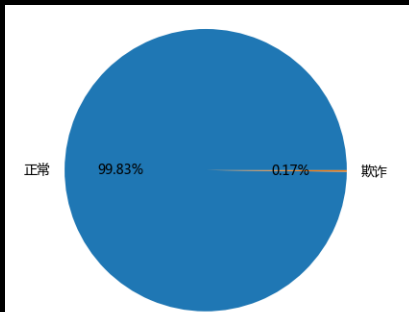
	Time	V1	V2	V3	V4	V5	V6	V7
\								
0	0.0	-1.359807	-0.072781	2.536347	1.378155	-0.338321	0.462388	0.239599
1	0.0	1.191857	0.266151	0.166480	0.448154	0.060018	-0.082361	-0.078803
2	1.0	-1.358354	-1.340163	1.773209	0.379780	-0.503198	1.800499	0.791461
3	1.0	-0.966272	-0.185226	1.792993	-0.863291	-0.010309	1.247203	0.237609
4	2.0	-1.158233	0.877737	1.548718	0.403034	-0.407193	0.095921	0.592941
		V8	V9	...	V21	V22	V23	V24
0	0.098698	0.363787	...	-0.018307	0.277838	-0.110474	0.066928	
1	0.085102	-0.255425	...	-0.225775	-0.638672	0.101288	-0.339846	
2	0.247676	-1.514654	...	0.247998	0.771679	0.909412	-0.689281	
3	0.377436	-1.387024	...	-0.108300	0.005274	-0.190321	-1.175575	
4	-0.270533	0.817739	...	-0.009431	0.798278	-0.137458	0.141267	
		V25	V26	V27	V28	Amount	Class	
0	0.128539	-0.189115	0.133558	-0.021053	149.62	0		
1	0.167170	0.125895	-0.008983	0.014724	2.69	0		
2	-0.327642	-0.139097	-0.055353	-0.059752	378.66	0		
3	0.647376	-0.221929	0.062723	0.061458	123.50	0		
4	-0.206010	0.502292	0.219422	0.215153	69.99	0		

[5 rows x 31 columns]

算法的应用实战

信用卡欺诈行为的识别

```
# 读入数据
creditcard = pd.read_csv(r'creditcard.csv')
# 为确保绘制的饼图为圆形，需执行如下代码
plt.axes(aspect = 'equal')
# 统计交易是否为欺诈的频数
counts = creditcard.Class.value_counts()
# 绘制饼图
plt.pie(x = counts, # 绘图数据
        labels=pd.Series(counts.index).map({0:'正常',1:'欺诈'}), # 添加文字标签
        autopct='%.2f%%' # 设置百分比的格式，这里保留两位小数
    )
# 显示图形
plt.show()
```



算法的应用实战

信用卡欺诈行为的识别

```
# 删除自变量中的Time变量
X = creditcard.drop(['Time','Class'], axis = 1)
y = creditcard.Class

# 将数据拆分为训练集和测试集
X_train,X_test,y_train,y_test = model_selection.train_test_split(X,y,test_size = 0.3, random_state = 1234)
```

算法的应用实战

信用卡欺诈行为的识别

```
# 导入第三方包
from imblearn.over_sampling import SMOTE

# 运用SMOTE算法实现训练数据集的平衡
over_samples = SMOTE(random_state=1234)
over_samples_X, over_samples_y = over_samples.fit_sample(X_train, y_train)

# 重抽样前的类别比例
print(y_train.value_counts()/len(y_train))

# 重抽样后的类别比例
print(pd.Series(over_samples_y).value_counts()/len(over_samples_y))
```

out:

```
0    0.998249
1    0.001751
Name: Class, dtype: float64
1    0.5
0    0.5
```

算法的应用实战

信用卡欺诈行为的识别

```
# 导入第三方包
import xgboost
import numpy as np

# 构建XGBoost分类器
xgboost = xgboost.XGBClassifier()

# 使用重抽样后的数据, 对其建模
xgboost.fit(over_samples_X, over_samples_y)

# 将模型运用到测试数据集中
resample_pred = xgboost.predict(np.array(X_test))
```

算法的应用实战

信用卡欺诈行为的识别

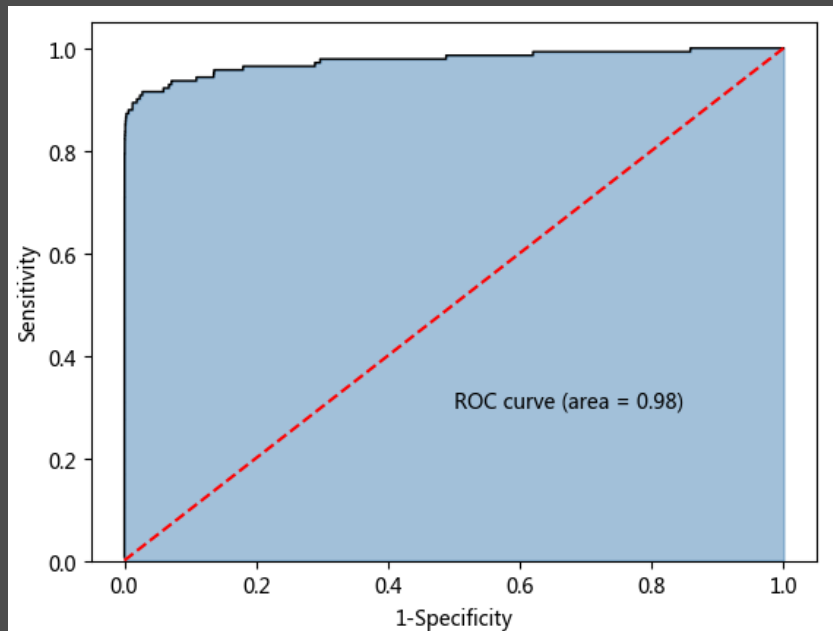
```
# 返回模型的预测效果
print('模型的准确率为: \n', metrics.accuracy_score(y_test, resample_pred))
print('模型的评估报告: \n', metrics.classification_report(y_test, resample_pred))
```

out:

模型的准确率为:
0.990145477102

算法的应用实战

信用卡欺诈行为的识别



算法的应用实战

信用卡欺诈行为的识别

```
# 构建XGBoost分类器
xgboost2 = xgboost.XGBClassifier()

# 使用非平衡的训练数据集拟合模型
xgboost2.fit(X_train,y_train)

# 基于拟合的模型对测试数据集进行预测
pred2 = xgboost2.predict(X_test)
```

算法的应用实战

信用卡欺诈行为的识别

```
# 返回模型的预测效果
print('模型的准确率为: \n',metrics.accuracy_score(y_test, pred2))
print('模型的评估报告: \n',metrics.classification_report(y_test, pred2))
```

out:

模型的准确率为:
0.999403110846

算法的应用实战

信用卡欺诈行为的识别

